TCS NQT 2027 · SECTION 5 OF 5 · PART B · ADVANCED

Advanced Coding Complete Guide

Every topic explained from zero: rules, formulas, diagrams, solved examples and practice questions.

16	63+	128+	20
topics from basic to advanced	solved examples	practice questions	question mini test

ONLINE LEARNING MOCK TEST SERIES · ATC DEVELOPERS

About this guide

Advanced Coding is the last section of the TCS NQT and has 2 problems in 90 minutes: one Easy and one Medium. Languages allowed: C, C++, Java, Python and Perl. Every code example in this guide was run on the sample input shown, and the sample outputs are real.

How each topic is organised

- **Start from zero:** the idea in plain words.
- **Formulas and rules:** a yellow box to revise quickly.
- **Diagrams:** pictures, charts and tables.
- **Solved examples:** step by step.
- **Shortcuts and mistakes** to avoid.
- **Practice questions:** Basic, Medium and Advanced. Answers are at the end.

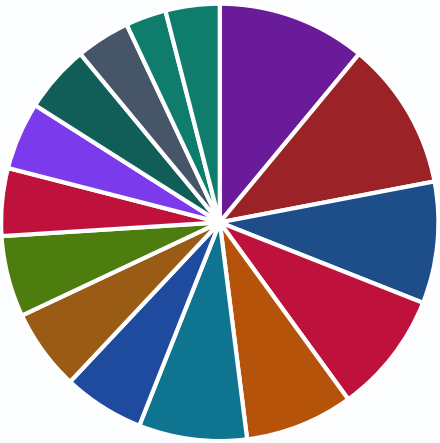
How to study

- Read one topic, then solve its practice set without looking at the answers.
- Check answers and note wrong ones in an error log.
- Revise the formula boxes daily.
- After all topics, take the mini test under the real time limit.
- Follow the priority: do **Must-do** topics first.

Topic priority and contents

1. Programming Fundamentals and Input/Output	Must-do	2. Time Complexity and Constraints	Must-do
3. Arrays	Must-do	4. Strings	Must-do
5. Math and Number Theory	Must-do	6. Sorting and Searching	Must-do
7. Hashing, Sets and Maps	Must-do	8. Two Pointers and Sliding Window	Important
9. Recursion and Backtracking	Important	10. Stack and Queue	Important
11. Matrix and 2D Arrays	Important	12. Greedy Algorithms	Important
13. Dynamic Programming	Must-do	14. Linked Lists, Trees and Graphs	Good to know
15. Bit Manipulation	Good to know	16. Exam Strategy, Edge Cases and Debugging	Must-do

Where to spend your practice time

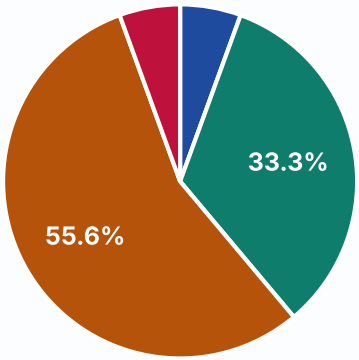


- Arrays 11%
- Strings 11%
- Math and Number Theory 9%
- Dynamic Programming 9%
- Sorting and Searching 8%
- Hashing 8%
- Programming Fundamentals 6%
- Greedy 6%
- Two Pointers and Window 6%
- Recursion and Backtracking 5%
- Stack and Queue 5%
- Matrix 5%

Suggested share of practice time per topic. This is our preparation guidance, not an official TCS weightage.

Section strategy

Advanced Coding has 2 problems in 90 minutes: one Easy (about 35 minutes) and one Medium (about 55 minutes). It decides the Digital and Prime roles, and your score depends on how many hidden test cases pass. Allowed languages: C, C++, Java, Python and Perl.



- Read and plan (5 min)
- Easy problem (30 min)
- Medium problem (50 min)
- Test and debug (5 min)

90 minutes in total

Easy ≈ 35 min, Medium ≈ 55 min

Always submit something that passes the sample tests.

A sensible split of the 90 minutes.

How scoring works

- Each problem is run on many hidden test cases.
- You get credit for the cases that pass.
- A slow solution may pass small tests and fail large ones.
- Partial solutions still score.

Which language?

- **Python:** shortest code, easiest for beginners, slower.
- **C++:** fastest, needs care with overflow.
- **Java:** safe and verbose.
- Pick one and stay with it for all 90 days.

Winning habits

- Write brute force first.
- Read the constraints before choosing the method.
- Test edge cases before submitting.
- Keep templates for input, gcd, sieve, BFS and DP.

Where marks are lost

- Wrong input or output format.
- Integer overflow.
- Too slow for the largest tests.
- No handling of edge cases.

ADVANCED CODING · MUST-DO

01 Programming Fundamentals and Input/Output

Every coding problem starts with reading input correctly and ends with printing exactly the required output. Many solutions fail not because the logic is wrong but because of a mistake in reading, data types or the output format.

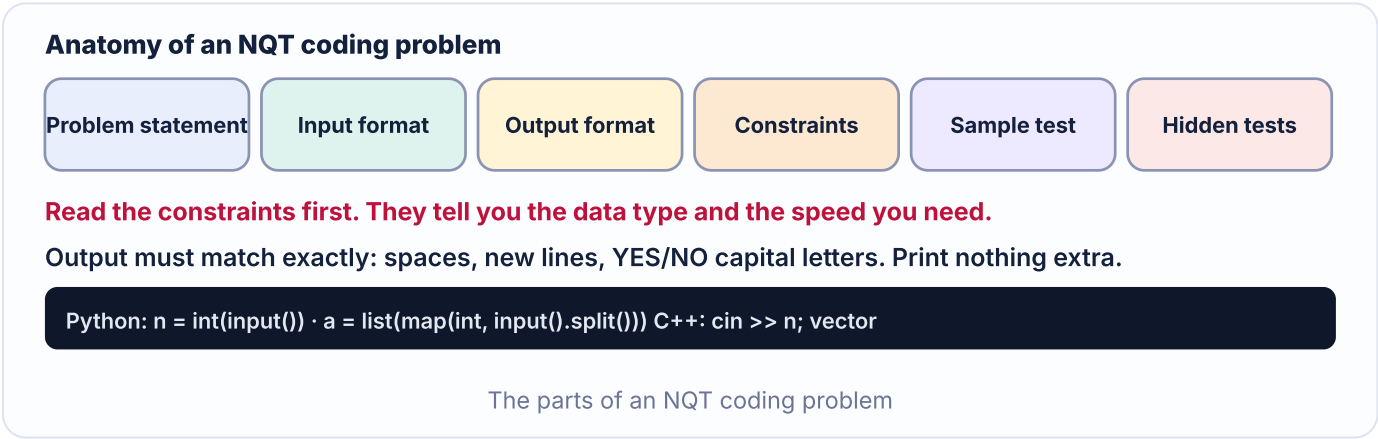
Start from zero

Anatomy of a problem

A TCS NQT coding problem has a statement, an input format, an output format, constraints, one or two sample tests and many hidden tests. Your code is judged on the hidden tests. Passing the sample is only the start.

Reading input

Python: `n = int(input())` reads one integer; `a = list(map(int, input().split()))` reads a line of integers. C++: `cin >> n;` and a loop to read an array. Java: `Scanner sc = new Scanner(System.in);` then `sc.nextInt()`. For many test cases the first line is T and the rest repeats T times.



The parts of an NQT coding problem

Data types and overflow

In C, C++ and Java, `int` holds about ±2.1 × 10^9. Use `long long` (C++) or `long` (Java) when sums or products can reach 10^{10} or more. Python integers are unbounded, so overflow is not a problem there. Many problems ask for an answer modulo $10^9 + 7$ to keep numbers small.

Operators you must know

Integer division `//` (Python) or `/` on ints (C++); remainder `%`. Python's `%` gives a non-negative result for a positive divisor even if the dividend is negative, but C++ can give a negative remainder. Comparison and logical operators: ==, `!=`, `and/or/not` (Python) or `&& || !` (C++).

Loops and patterns

Use a `for` loop when the count is known and a `while` loop otherwise. Patterns (triangles, pyramids) are nested loops: the outer loop is the row and the inner loop prints the columns. Always check the output format: spaces and line breaks matter.

Output rules

Print exactly what is asked. Capital letters for YES and NO if the sample uses them. No extra text such as 'Answer:'. For several values on one line use spaces; for several test cases use one line per case unless

told otherwise.

Formula and rule box

- int ≈ ±2.1 × 10^9; long long ≈ ±9.2 × 10^{18}
- Use long long when a product can exceed 2 × 10^9
- (a × b) mod m = ((a mod m) × (b mod m)) mod m
- Multiple tests: read T, then loop T times
- Digit sum: repeat n % 10 and n // 10
- Leap year: divisible by 4 and (not by 100 or divisible by 400)

Step-by-step method

1. Read the statement, input format and constraints.
2. Choose data types from the constraints.
3. Write the logic for the sample and trace it by hand.
4. Test edge cases (0, 1, maximum, negative values) before submitting.

Solved examples

Example 1. Read T test cases. For each, read two integers and print their sum.

Solution: Read T first, then loop T times, read a and b and print a + b. In C++ use long long to be safe.

```python
t = int(input())
for _ in range(t):
    a, b = map(int, input().split())
    print(a + b)
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int t; cin >> t;
    while (t--) {
        long long a, b; cin >> a >> b;
        cout << a + b << "\n";
    }
    return 0;
}
```

Sample input: `3 / 1 2 / 10 20 / -5 5` **Sample output:** `3 / 30 / 0`

Answer: Output: 3 / 30 / 0

Example 2. Print a right triangle of stars of height n.

Solution: Row i prints i stars. A string multiplied by a number repeats the string.

```python
n = int(input())
for i in range(1, n + 1):
    print('*' * i)
```

Sample input: `4` **Sample output:** `* / ** / *** / ****`

Answer: Output: * / ** / * / ****

Example 3. Print the numbers 1 to n, but 'Fizz' for multiples of 3, 'Buzz' for multiples of 5 and 'FizzBuzz' for multiples of both, separated by spaces.

Solution: Check the multiple of 15 first, otherwise 3 and 5 will catch it early.

```python
n = int(input())
out = []
for i in range(1, n + 1):
    if i % 15 == 0: out.append('FizzBuzz')
    elif i % 3 == 0: out.append('Fizz')
    elif i % 5 == 0: out.append('Buzz')
    else: out.append(str(i))
print(*out)
```

Sample input: `15` **Sample output:** `1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz`

Answer: Output: 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz

Example 4. Read a and b (each up to 10^9). Print (a × b) mod 1,000,000,007.

Solution: The product reaches 10^{18}. In C++ it needs long long; in Python it is automatic. Take the modulo after multiplying.

```python
a, b = map(int, input().split())
MOD = 10**9 + 7
print(a * b % MOD)
```

```cpp
#include <iostream>
using namespace std;
int main() {
    long long a, b, MOD = 1000000007;
    cin >> a >> b;
    cout << (a % MOD) * (b % MOD) % MOD << endl;
    return 0;
}
```

Sample input: `1000000000 1000000000` **Sample output:** `49`

Answer: Output: 49

Shortcuts

- Python's print(*list) prints items separated by spaces.
- Use long long by default in C++ when unsure.
- Trace the sample by hand before running.

Common mistakes

- Forgetting to read T test cases.
- Using int where the answer needs long long.
- Printing extra spaces or text.

Practice questions: Programming Fundamentals and Input/Output

1. Print the sum of digits of n. *Sample input:* `4825` Basic
2. Print YES if the year is a leap year, otherwise NO. *Sample input:* `2100` Basic
3. Print the sum of the first n natural numbers. *Sample input:* `100` Basic
4. Print the number formed by reversing the digits of n (drop leading zeros). *Sample input:* `1200` Basic
5. Print the count of digits in n. *Sample input:* `12345` Medium
6. Print the largest of three numbers. *Sample input:* `7 19 4` Medium
7. Print n! modulo 1,000,000,007. *Sample input:* `20` Medium
8. Print how many numbers from 1 to n are divisible by 3 or 5. *Sample input:* `20` Medium

Answers: see the answer key at the end of this guide.

02 ADVANCED CODING · MUST-DO
Time Complexity and Constraints

A correct but slow solution scores zero on large hidden tests. Complexity tells you in advance whether your idea will finish in time. Read the constraints before writing any code.

Start from zero

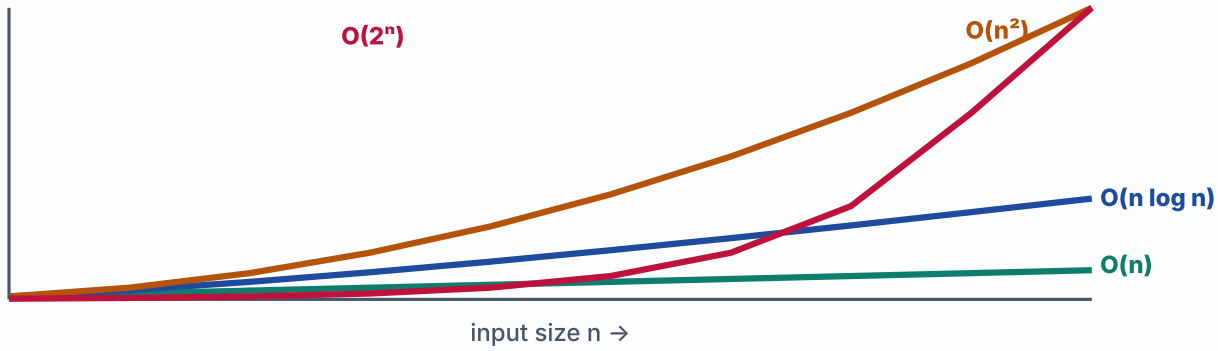
What Big-O means

Big-O describes how the running time grows when the input size n grows, ignoring constants. A single loop over n items is O(n). A loop inside a loop is O(n^2). Halving the range each step (binary search) is O(log n). Sorting is O(n log n).

How many operations fit in the time limit

A typical judge allows about 10^8 simple operations per second in C++ and about 10^7 in Python. So with n = 10^5, an O(n^2) solution does 10^{10} operations and fails, but O(n log n) does under 2 × 10^6 and passes.

How running time grows with the input



Growth curves of common complexities

Match constraints to algorithms

Use the table: n ≤ 10 allows brute force over permutations; n ≤ 20 allows subsets; n ≤ 500 allows O(n^3); n ≤ 5,000 allows O(n^2); n ≤ 10^5 needs O(n log n); n ≤ 10^7 needs O(n); n up to 10^{18} needs O(log n) such as binary search or fast power.

Cost of common operations

List append O(1); list.pop(0) or insert at the front O(n); dictionary/set lookup O(1) on average; sorted() O(n log n); string concatenation in a loop can be O(n^2) (use join); slicing a list copies it (O(n)).

Space complexity

Extra memory used: O(1) for a few variables, O(n) for an extra array. A 2D table of 10^4 × 10^4 is too large; reduce it to one or two rows when possible.

Optimise step by step

First write a brute force that is surely correct. Then find the repeated work: nested loops that recompute sums (use prefix sums), searches (use a set or binary search) or recursion that repeats states (use memoisation).

$n \le 10$	**$O(n!)$**	brute force, permutations
$n \le 20$	**$O(2^n)$**	subsets, bitmask
$n \le 500$	**$O(n^3)$**	triple loops, Floyd
$n \le 5{,}000$	**$O(n^2)$**	double loop, simple DP
$n \le 10^5$–10^6	**$O(n \log n)$**	sorting, binary search, heap
$n \le 10^7$	**$O(n)$**	single pass, hashing
$n \le 10^{18}$	**$O(\log n)$**	binary search, fast power

Match the constraint to an algorithm

Formula and rule box

- $\approx 10^8$ operations/second (C++), $\approx 10^7$ (Python)
- $n \le 10^5 \rightarrow O(n \log n)$ or better
- $n \le 5{,}000 \rightarrow O(n^2)$ is fine
- $n \le 20 \rightarrow O(2^n)$ is fine
- sort = $O(n \log n)$; binary search = $O(\log n)$
- dict/set lookup = $O(1)$ average

Step-by-step method

1. Read the constraint on n.
2. Compute operations for your idea (n^2 for nested loops).
3. If too slow, find the repeated work.
4. Replace it with a prefix sum, a hash map, sorting or binary search.

Solved examples

Example 1. Given an array and a target, print YES if two different elements sum to the target. First line n, second line the array, third line target.

Solution: Brute force checks all pairs in O(n²). Using a set of seen values finds the partner in O(1), giving O(n) overall.

```python
n = int(input())
a = list(map(int, input().split()))
target = int(input())
seen = set()
found = False
for x in a:
    if target - x in seen:
        found = True
        break
    seen.add(x)
print('YES' if found else 'NO')
```

```cpp
#include <iostream>
#include <unordered_set>
using namespace std;
int main() {
    int n; cin >> n;
    long long a[100000];
    for (int i = 0; i < n; i++) cin >> a[i];
    long long target; cin >> target;
    unordered_set<long long> seen;
    bool found = false;
    for (int i = 0; i < n; i++) {
        if (seen.count(target - a[i])) { found = true; break; }
        seen.insert(a[i]);
    }
    cout << (found ? "YES" : "NO") << endl;
    return 0;
}
```

Sample input: `4 / 2 7 11 15 / 9` **Sample output:** YES

Answer: Output: YES

Example 2. Print the sum of all subarray sums of an array of length n (n up to 10^5).

Solution: Each element a[i] appears in (i+1) × (n−i) subarrays. So the total is the sum of a[i] × (i+1) × (n−i), computed in O(n) instead of O(n^3).

```python
n = int(input())
a = list(map(int, input().split()))
total = 0
for i, x in enumerate(a):
    total += x * (i + 1) * (n - i)
print(total)
```

Sample input: `3 / 1 2 3` **Sample output:** `20`

Answer: Output: 20

Example 3. Count primes up to n (n up to 10^6).

Solution: Checking every number by trial division is too slow. A sieve marks multiples of each prime in O(n log log n).

```python
n = int(input())
is_p = [True] * (n + 1)
is_p[0] = False
if n >= 1: is_p[1] = False
i = 2
while i * i <= n:
    if is_p[i]:
        for j in range(i * i, n + 1, i):
            is_p[j] = False
    i += 1
print(sum(is_p))
```

Sample input: `100` **Sample output:** `25`

Answer: Output: 25

Shortcuts

- Memorise the constraint-to-complexity table.
- If n ≤ 10^5, avoid double loops over the array.
- Use sets and dictionaries to replace inner loops that search.

Common mistakes

- Writing nested loops when n is 10^5 or larger.
- Rebuilding strings with + in a long loop.
- Using list.pop(0) in a loop.

Practice questions: Time Complexity and Constraints

1. n = 2 × 10^5 items. Which time complexity do you need? Basic
2. An O(n^2) idea for n = 10^5: about how many operations? Basic
3. Which structure gives O(1) average lookup for 'seen before'? Basic
4. What is the complexity of sorting an array of n items? Basic
5. What is the complexity of binary search on a sorted array? Medium
6. n ≤ 20 and you must try every subset. Complexity? Medium

7. Why is list.pop(0) in a loop slow? Medium

8. Which method finds the sum of any subarray in O(1) after O(n) preparation? Medium

Answers: see the answer key at the end of this guide.

03 ADVANCED CODING · MUST-DO
Arrays

Arrays are the most common topic in the coding round. Most problems ask you to traverse the array once, remember something while you go, and print the result.

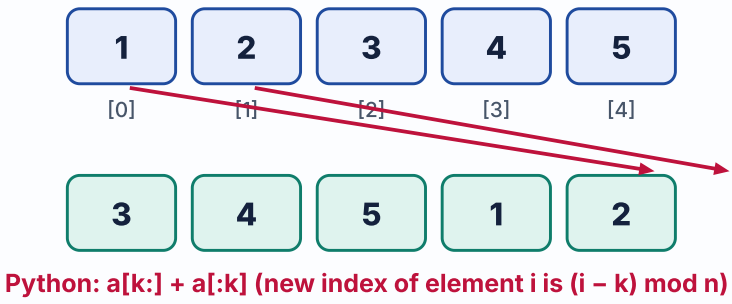
Start from zero

What an array is

An array is a list of items stored in order, each with an index starting at 0. Access by index is O(1). Traversal visits each item once: O(n). In Python the same structure is a list.

Traversal patterns

Single pass for sum, maximum, minimum and counting. Two passes when you need a total first and then a comparison. Keep running variables (best so far, count so far) instead of storing everything.



Array rotation and prefix sums

Prefix sums

Build P where P[i] is the sum of the first i elements. Then the sum of elements l to r is P[r + 1] − P[l] in O(1). This turns many range-sum queries from O(n) each into O(1) each.

Kadane's algorithm

Maximum sum of a contiguous subarray in one pass: keep a running sum that restarts at 0 when it becomes negative, and track the best sum seen. Handle the all-negative case by starting with the first element.

Rotation and reversal

Rotate left by k: `a[k:] + a[:k]` after k %= n. Reverse: `a[::-1]`. For in-place O(1)-space rotation, reverse the whole array and then the two parts.

Duplicates, missing numbers, frequency

A set tells you if there is a duplicate. For numbers 1 to n with one missing, the answer is n(n + 1)/2 − sum(array). A dictionary or counter array counts frequencies. Sorting first often simplifies pairing problems.

Formula and rule box

- P[i] = a[0] + … + a[i−1]; sum(l..r) = P[r+1] − P[l]
- Kadane: cur = max(x, cur + x); best = max(best, cur)
- Missing number from 1..n: n(n+1)/2 − sum
- Rotate left by k: a[k:] + a[:k] (k %= n)
- Sum of first n numbers: n(n+1)/2

Step-by-step method

1. Decide: single pass, prefix sum, sorting or hashing?
2. Track the running value (sum, best, count).
3. Handle n = 1 and all-negative or all-equal arrays.
4. Check indices (off-by-one) on the sample.

Solved examples

Example 1. Rotate an array left by k positions. Print the new array.

Solution: Reduce k modulo n, then join the slice from k to the end with the slice before k.

```python
n, k = map(int, input().split())
a = list(map(int, input().split()))
k %= n
print(*(a[k:] + a[:k]))
```

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    k %= n;
    for (int i = 0; i < n; i++) cout << a[(i + k) % n] << (i + 1 < n ? " " : "\n");
    return 0;
}
```

Sample input: `5 2 / 1 2 3 4 5` **Sample output:** `3 4 5 1 2`

Answer: Output: 3 4 5 1 2

Example 2. Print the maximum sum of a contiguous subarray.

Solution: Kadane's algorithm: extend the running sum or restart at the current element.

```python
n = int(input())
a = list(map(int, input().split()))
cur = best = a[0]
for x in a[1:]:
    cur = max(x, cur + x)
    best = max(best, cur)
print(best)
```

```cpp
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int n; cin >> n;
    long long a, cur, best;
    cin >> a; cur = best = a;
    for (int i = 1; i < n; i++) {
        cin >> a;
        cur = max(a, cur + a);
        best = max(best, cur);
    }
    cout << best << endl;
    return 0;
}
```

Sample input: `9 / -2 1 -3 4 -1 2 1 -5 4` **Sample output:** `6`

Answer: Output: 6

Example 3. Answer q range-sum queries. Input: n, the array, q, then q lines with l and r (1-indexed).

Solution: Build prefix sums once. Each query is P[r] − P[l − 1].

```python
n = int(input())
a = list(map(int, input().split()))
P = [0]
for x in a:
    P.append(P[-1] + x)
q = int(input())
for _ in range(q):
    l, r = map(int, input().split())
    print(P[r] - P[l - 1])
```

Sample input: `5 / 1 2 3 4 5 / 2 / 1 3 / 2 5` **Sample output:** `6 / 14`

Answer: Output: 6 / 14

Example 4. An array of n − 1 distinct numbers from 1 to n has one number missing. Find it.

Solution: The total of 1 to n is n(n + 1)/2. Subtract the array's sum.

```python
n = int(input())
a = list(map(int, input().split()))
print(n * (n + 1) // 2 - sum(a))
```

Sample input: `5 / 1 2 4 5` **Sample output:** 3

Answer: Output: 3

Shortcuts

- Use prefix sums whenever you see many range queries.
- Kadane works in one pass and O(1) space.
- Sort when the problem is about pairs, gaps or the k-th element.

Common mistakes

- Starting Kadane with 0 and failing for all-negative arrays.
- Forgetting k %= n in rotation.
- Off-by-one in prefix sum queries.

Practice questions: Arrays

1. Print the second largest distinct element. *Sample input:* `6 / 4 9 1 9 7 3` **Basic**
2. Print the number of pairs (i < j) whose sum equals target. *Sample input:* `5 / 1 5 7 -1 5 / 6` **Basic**
3. Move all zeros to the end, keeping the order of other elements. *Sample input:* `7 / 0 1 0 3 12 0 4` **Basic**
4. Print the maximum product of any two elements. *Sample input:* `5 / -10 -3 5 6 2` **Basic**
5. Print all leaders (elements greater than everything to their right), in original order. *Sample input:* `6 / 16 17 4 3 5 2` **Medium**
6. Rotate the array right by k positions. *Sample input:* `5 2 / 1 2 3 4 5` **Medium**
7. Print the length of the longest strictly increasing contiguous run. *Sample input:* `8 / 1 3 5 4 7 8 9 2` **Medium**
8. Print the duplicate number in an array of n + 1 numbers from 1 to n. *Sample input:* `6 / 3 1 3 4 2 5` **Medium**

Answers: see the answer key at the end of this guide.

04 ADVANCED CODING · MUST-DO
Strings

Strings are arrays of characters, and string problems are about scanning, counting and comparing. Most solutions use one of four tools: two pointers, a frequency count, a stack or a simple pass with a running state.

Start from zero

Basics

A string is indexed from 0. In Python strings are immutable: `s[i] = 'a'` is not allowed, so build a new string or use a list of characters. Useful methods: `s.lower()`, `s.upper()`, `s.isalpha()`, `s.isdigit()`, `s.split()`, `' '.join(list)`, `s[::-1]` for reversing.

Palindromes

A string equal to its reverse. Compare s with s[::-1], or use two pointers moving inwards. For problems that ignore case and punctuation, first keep only letters and digits and convert to lower case.

Two pointers check a palindrome: compare the ends and move inwards



Frequency count: use an array of 26 counters or a dictionary. Anagram ⇔ equal counters.

Compression: aaabb → a3b2 (count runs of equal characters)

Two pointers on a palindrome

Frequency and anagrams

Count characters with a dictionary or a 26-slot array. Two words are anagrams if their counts are equal (or their sorted letters are equal). The first non-repeating character is found by counting once and scanning again.

Run-length compression and parsing

Scan the string, count how many equal characters are in a row, and output the character with the count. Parsing numbers from text: loop over characters and build the number while you see digits.

Substrings and subsequences

A substring is contiguous; a subsequence keeps order but may skip. Count of substrings of length n is n(n + 1)/2. For longest substring problems use a sliding window.

Common pitfalls

Strings with spaces: use `input()` rather than split. Case: 'A' ≠ 'a'. Empty strings. Python string concatenation inside a long loop is slow; collect pieces in a list and join once.

Formula and rule box

- s[::-1] reverses a string
- Anagram ⇔ equal character counts
- Number of substrings = n(n + 1)/2
- ord('a') = 97; ord(c) − 97 gives 0–25
- Palindrome: s == s[::-1]
- Join pieces: ''.join(list)

Step-by-step method

1. Decide whether you need counting, two pointers or a scan.
2. Normalise (case, spaces) if the problem requires it.
3. Handle empty and single-character strings.
4. Print in the exact format (YES/NO or the string).

Solved examples

Example 1. Print YES if the string is a palindrome ignoring case and non-alphanumeric characters, else NO.

Solution: Clean the string, then compare it with its reverse.

```python
s = input()
t = ''.join(c.lower() for c in s if c.isalnum())
print('YES' if t == t[::-1] else 'NO')
```

```cpp
#include <iostream>
#include <string>
#include <cctype>
using namespace std;
int main() {
    string s, t;
    getline(cin, s);
    for (char c : s) if (isalnum((unsigned char)c)) t += tolower(c);
    int i = 0, j = (int)t.size() - 1; bool ok = true;
    while (i < j) { if (t[i++] != t[j--]) { ok = false; break; } }
    cout << (ok ? "YES" : "NO") << endl;
    return 0;
}
```

Sample input: `A man, a plan, a canal: Panama` **Sample output:** `YES`

Answer: Output: YES

Example 2. Print YES if two words are anagrams of each other.

Solution: Sort the letters of both words and compare, or compare frequency counts.

```python
a = input().strip().lower()
b = input().strip().lower()
print('YES' if sorted(a) == sorted(b) else 'NO')
```

Sample input: `Listen / Silent` **Sample output:** `YES`

Answer: Output: YES

Example 3. Compress a string with run-length encoding: aaabbc becomes a3b2c1.

Solution: Scan with a counter. When the character changes, output the previous character and its count.

```python
s = input().strip()
out = []
i = 0
while i < len(s):
    j = i
    while j < len(s) and s[j] == s[i]:
        j += 1
    out.append(s[i] + str(j - i))
    i = j
print(''.join(out))
```

Sample input: `aaabbc` **Sample output:** `a3b2c1`

Answer: Output: a3b2c1

Example 4. Print the first non-repeating character, or -1 if none exists.

Solution: Count each character first, then scan the string in order for a count of 1.

```python
s = input().strip()
cnt = {}
for c in s:
    cnt[c] = cnt.get(c, 0) + 1
ans = '-1'
for c in s:
    if cnt[c] == 1:
        ans = c
        break
print(ans)
```

Sample input: `swiss` **Sample output:** `w`

Answer: Output: w

Shortcuts

- sorted(a) == sorted(b) is the quickest anagram test.
- Use a dictionary or collections.Counter for frequency.
- isalnum() filters letters and digits.

Common mistakes

- Forgetting to ignore case when the problem says to.
- Using split() on a line with spaces you need to keep.
- Building long strings with += inside a loop in Python.

Practice questions: Strings

1. Print the number of vowels in the string. *Sample input:* `Programming` **Basic**
2. Reverse each word of the sentence, keeping the word order. *Sample input:* `hello world` **Basic**
3. Print the most frequent character (smallest if tied). *Sample input:* `banana` **Basic**
4. Print the length of the longest common prefix of two words. *Sample input:* `flower / flow` **Basic**
5. Print YES if the string contains every letter a to z (a pangram). *Sample input:* `the quick brown fox jumps over the lazy dog` **Medium**
6. Print the number of words in the sentence. *Sample input:* `TCS NQT is easy to crack` **Medium**
7. Print the string with each letter shifted by 3 (a→d, z→c), lowercase only. *Sample input:* `xyz` `abc` **Medium**
8. Print the length of the longest substring without repeating characters. *Sample input:* `abcabcbb` **Medium**

Answers: see the answer key at the end of this guide.

05 ADVANCED CODING · MUST-DO
Math and Number Theory

Number problems appear in almost every test. The tools are few: divisibility, the greatest common divisor, primes, fast exponentiation and digit manipulation.

Start from zero

Prime numbers

A prime has exactly two factors. To test n, try divisors from 2 up to √n; if none divides n, it is prime (n > 1). To list all primes up to n use the sieve of Eratosthenes: start from 2, cross out its multiples, then the next uncrossed number, and so on.

GCD and LCM

Euclid's algorithm: gcd(a, b) = gcd(b, a mod b), stopping when b is 0. LCM(a, b) = a × b / gcd(a, b). Python has `math.gcd`. For several numbers fold the function over the list.

Sieve of Eratosthenes up to 30: cross out multiples of 2, 3, 5

2	**3**	4	**5**	6	**7**	8	9	10	**11**	12	**13**	14	15	16
17	18	**19**	20	21	22	**23**	24	25	26	27	28	**29**	30	

Primes: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29. Start at p^2, stop at √n. Time O(n log log n).

The sieve of Eratosthenes up to 30

Divisors and factorisation

Factors come in pairs (d, n/d), so loop d from 1 to √n. The number of divisors comes from the prime factorisation: if n = p^a × q^b then divisors = (a + 1)(b + 1). A perfect number equals the sum of its proper divisors (6, 28, 496).

Digits and number properties

Use n % 10 for the last digit and n // 10 to remove it. Armstrong numbers equal the sum of their digits each raised to the number of digits (153 = $1^3 + 5^3 + 3^3$). A palindromic number reads the same reversed.

Fast exponentiation

To compute a^b mod m quickly use squaring: if b is even, a^b = (a^2)^(b/2); if b is odd, a^b = a × a^(b−1). In Python use `pow(a, b, m)`. This runs in O(log b).

Base conversion and other formulas

Decimal to binary: repeatedly divide by 2 and read the remainders backwards; Python: `bin(n)`. Fibonacci: F(n) = F(n−1) + F(n−2). Trailing zeros in n! = n//5 + n//25 + n//125 + ...

Formula and rule box

- Prime test: check divisors up to √n
- gcd(a, b) = gcd(b, a mod b)
- lcm(a, b) = a × b / gcd(a, b)
- pow(a, b, m) computes a^b mod m
- Divisors of p^a q^b: (a + 1)(b + 1)
- Trailing zeros of n! = n//5 + n//25 + n//125 + ...
- Sum 1..n = n(n + 1)/2

Step-by-step method

1. Identify the property (prime, divisor, digit, power).
2. Choose the standard routine (√n loop, Euclid, sieve, fast power).
3. Check the constraint to pick the right efficiency.
4. Test on 0, 1, 2 and a large value.

Solved examples

Example 1. Print YES if n is prime, otherwise NO.

Solution: Numbers below 2 are not prime. Test divisors up to √n.

```python
import math
n = int(input())
ok = n >= 2
for d in range(2, math.isqrt(n) + 1):
    if n % d == 0:
        ok = False
        break
print('YES' if ok else 'NO')
```

```cpp
#include <iostream>
using namespace std;
int main() {
    long long n; cin >> n;
    bool ok = n >= 2;
    for (long long d = 2; d * d <= n; d++)
        if (n % d == 0) { ok = false; break; }
    cout << (ok ? "YES" : "NO") << endl;
    return 0;
}
```

Sample input: 97 **Sample output:** YES

Answer: Output: YES

Example 2. Print the gcd and lcm of two numbers.

Solution: Use Euclid's algorithm for gcd and the formula lcm = a × b / gcd.

```python
a, b = map(int, input().split())
def gcd(x, y):
    while y:
        x, y = y, x % y
    return x
g = gcd(a, b)
print(g, a * b // g)
```

```cpp
#include <iostream>
using namespace std;
long long gcd(long long x, long long y) { return y ? gcd(y, x % y) : x; }
int main() {
    long long a, b; cin >> a >> b;
    long long g = gcd(a, b);
    cout << g << " " << a / g * b << endl;
    return 0;
}
```

Sample input: `12 18` **Sample output:** `6 36`

Answer: Output: 6 36

Example 3. Print a^b mod m using fast exponentiation (a, b up to 10^9, m up to $10^9 + 7$).

Solution: Square the base and halve the exponent each step.

```python
a, b, m = map(int, input().split())
res = 1
a %= m
while b:
    if b & 1:
        res = res * a % m
    a = a * a % m
    b >>= 1
print(res)
```

Sample input: `2 10 1000000007` **Sample output:** `1024`

Answer: Output: 1024

Example 4. Print the number of divisors of n.

Solution: Loop d to √n. Each divisor d pairs with n/d (count once if they are equal).

```python
n = int(input())
cnt = 0
d = 1
while d * d <= n:
    if n % d == 0:
        cnt += 1 if d * d == n else 2
    d += 1
print(cnt)
```

Sample input: `360` **Sample output:** `24`

Answer: Output: 24

Shortcuts

- math.isqrt(n) avoids floating-point errors.
- pow(a, b, m) is built into Python.
- Use the sieve when you need many primes.

Common mistakes

- Treating 1 as prime.
- Looping to n instead of √n.
- Overflow of a × b in C++ before taking the modulo.

Practice questions: Math and Number Theory

1. Print how many primes are at most n. *Sample input:* `50` Basic
2. Print the nth Fibonacci number with F(1) = F(2) = 1. *Sample input:* `10` Basic
3. Print the gcd of all numbers in the list. *Sample input:* `24 36 60` Basic
4. Print YES if n is an Armstrong number. *Sample input:* `153` Basic
5. Print the number of divisors of n. *Sample input:* `100` Medium
6. Print n in binary (without 0b). *Sample input:* `45` Medium
7. Print YES if n is a perfect number. *Sample input:* `28` Medium
8. Print the number of trailing zeros in n!. *Sample input:* `100` Medium

Answers: see the answer key at the end of this guide.

06 ADVANCED CODING · MUST-DO
Sorting and Searching

Sorting arranges data so that comparing, pairing and searching become easy. Binary search then finds items in a sorted list in logarithmic time. Together they solve a large share of medium problems.

Start from zero

Sorting in practice

Use the built-in sort: `a.sort()` or `sorted(a)` in Python, `sort(a.begin(), a.end())` in C++. Both run in O(n log n). Know bubble, selection and insertion sort for theory ($O(n^2)$) and merge sort and quick sort (O(n log n)).

Custom ordering

Sort by a key: `sorted(students, key=lambda x: (-x[1], x[0]))` sorts by marks descending, then by name ascending. In C++ pass a comparator function. Sorting by multiple keys is a favourite question.

Binary search for 23 in a sorted array: halve the range each time



lo=0, hi=9 → mid=4 (16 < 23) → lo=5 → mid=7 (56 > 23) → hi=6 → mid=5 → found at index 5

Each step halves the search space: about $\log_2(n)$ steps (1,000,000 items need ~20 steps).

Binary search halves the range

Binary search

On a sorted array keep a range [lo, hi]. Look at the middle: if it equals the target, done; if the target is larger, move lo to mid + 1; otherwise move hi to mid − 1. Each step halves the range, so it needs about $\log_2 n$ steps.

Lower bound and counting

The lower bound is the first index whose value is ≥ x. Python: `bisect.bisect_left(a, x)`. The count of x in a sorted array = bisect_right − bisect_left.

Searching on the answer

When 'the smallest value that works' is monotone (if x works, x + 1 works), binary search the answer itself and test each candidate with a simple check. This solves problems such as the minimum capacity or the smallest time.

Rotated sorted arrays

A sorted array rotated at some point still has one sorted half in every range. Compare the middle with the ends to decide which half is sorted and whether the target lies in it.

Formula and rule box

- Sorting: O(n log n)
- Binary search: O(log n)
- mid = lo + (hi − lo) // 2
- count(x) = bisect_right − bisect_left
- Sort key: (−marks, name) for marks descending
- Searching on the answer: check(mid) is monotone

Step-by-step method

1. Is the data sorted or can you sort it?
2. Pick key and order for sorting.
3. For search use binary search with a clear loop invariant.
4. Test with a single element and a target that is absent.

Solved examples

Example 1. Print the index (0-based) of a target in a sorted array, or -1 if absent.

Solution: Binary search: compare the middle with the target and discard half.

```python
n, x = map(int, input().split())
a = list(map(int, input().split()))
lo, hi, ans = 0, n - 1, -1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == x:
        ans = mid
        break
    if a[mid] < x:
        lo = mid + 1
    else:
        hi = mid - 1
print(ans)
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, x; cin >> n >> x;
    int a[100000];
    for (int i = 0; i < n; i++) cin >> a[i];
    int lo = 0, hi = n - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) { ans = mid; break; }
        if (a[mid] < x) lo = mid + 1; else hi = mid - 1;
    }
    cout << ans << endl;
    return 0;
}
```

Sample input: `10 23 / 2 5 8 12 16 23 38 56 72 91` **Sample output:** `5`

Answer: Output: 5

Example 2. Sort students by marks descending; ties by name ascending. Each line has a name and marks. Print the names.

Solution: Sort with a key of (−marks, name).

```python
n = int(input())
st = []
for _ in range(n):
    name, m = input().split()
    st.append((name, int(m)))
st.sort(key=lambda x: (-x[1], x[0]))
print(*[x[0] for x in st])
```

Sample input: `4 / Ravi 80 / Amit 90 / Zoya 80 / Bina 90` **Sample output:** `Amit Bina Ravi Zoya`

Answer: Output: Amit Bina Ravi Zoya

Example 3. Count how many times x appears in a sorted array.

Solution: Use bisect_left and bisect_right: their difference is the count.

```python
import bisect
n, x = map(int, input().split())
a = list(map(int, input().split()))
print(bisect.bisect_right(a, x) - bisect.bisect_left(a, x))
```

Sample input: `8 5 / 1 3 5 5 5 7 9 11` **Sample output:** `3`

Answer: Output: 3

Example 4. Print the k-th smallest element of an array.

Solution: Sort the array and take index k − 1.

```python
n, k = map(int, input().split())
a = sorted(map(int, input().split()))
print(a[k - 1])
```

Sample input: `6 3 / 7 10 4 3 20 15` **Sample output:** `7`

Answer: Output: 7

Shortcuts

- Use the built-in sort and bisect functions.
- mid = lo + (hi − lo) // 2 avoids overflow in C++.
- Sort first when the problem involves pairs or ranges.

Common mistakes

- Infinite loops from wrong lo/hi updates.
- Forgetting that binary search needs sorted data.
- Mixing ascending and descending in key functions.

Practice questions: Sorting and Searching

1. Print the third largest distinct number. *Sample input:* `8 / 12 35 1 10 34 1 35 12` Basic
2. Print the array sorted in ascending order. *Sample input:* `5 / 9 4 7 1 3` Basic
3. Print the smallest absolute difference between any two elements. *Sample input:* `5 / 12 3 9 20 7` Basic
4. Print the index of the first element that is greater than or equal to x in a sorted array (lower bound). *Sample input:* `6 7 / 1 3 5 7 9 11` Basic
5. Print the median of the array (average of two middles if n is even, print as a number). *Sample input:* `5 / 7 1 3 5 9` Medium
6. Print YES if the array has any duplicate. *Sample input:* `6 / 4 1 7 3 1 9` Medium
7. Find the minimum in a sorted array rotated at an unknown point. *Sample input:* `7 / 4 5 6 7 0 1 2` Medium
8. Print the integer square root of n using binary search. *Sample input:* 99 Medium

Answers: see the answer key at the end of this guide.

07 ADVANCED CODING · MUST-DO

Hashing, Sets and Maps

A hash map or set answers 'have I seen this before?' in constant time. It turns many $O(n^2)$ pair-searching problems into $O(n)$ problems, which is why hashing shows up so often.

Start from zero

Sets and dictionaries

A set stores unique values and supports fast membership tests (`x in s`). A dictionary stores key–value pairs (`d[key] = value`). In C++ use `unordered_set` and `unordered_map` (or `set` and `map` if you need sorted order).

Frequency maps

Count how often each value appears: `d[x] = d.get(x, 0) + 1` or `collections.Counter(a)`. Most problems about duplicates, majorities and anagrams are frequency problems.



Two-sum with a hash map

Two-sum and complements

For each element x, ask whether target − x has been seen. Store each element after the check so you do not pair an element with itself. One pass, O(n).

Prefix sums with a map

To count subarrays with sum K, keep a map from prefix sum to how many times it has occurred. At each position, add the count of (current prefix − K). This runs in O(n).

Grouping

Group items by a computed key: for anagram groups use the sorted word as the key. Use a dictionary of lists.

Longest consecutive sequence

Put all numbers in a set. For each number x with no x − 1 in the set, count up x + 1, x + 2 ... in the set. Total time O(n).

Formula and rule box

- Membership in set/dict: O(1) average
- Frequency: d[x] = d.get(x, 0) + 1
- Two-sum: look for target − x in seen
- Subarray sum K: count[prefix − K]
- Group key for anagrams: sorted(word)
- Counter(a).most_common(1) gives the most frequent item

Step-by-step method

1. Ask what you want to look up quickly.
2. Choose set (presence) or dictionary (count, index).
3. Decide the key (value, prefix sum, sorted word).
4. Update the map after querying it.

Solved examples

Example 1. Print the indices (0-based) of two numbers adding to the target.

Solution: Store each number's index. For each x, check if target − x is already stored.

```python
n, target = map(int, input().split())
a = list(map(int, input().split()))
seen = {}
for i, x in enumerate(a):
    if target - x in seen:
        print(seen[target - x], i)
        break
    seen[x] = i
```

```cpp
#include <iostream>
#include <unordered_map>
using namespace std;
int main() {
    int n, target; cin >> n >> target;
    unordered_map<int, int> seen;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        if (seen.count(target - x)) { cout << seen[target - x] << " " << i << endl;
return 0; }
        seen[x] = i;
    }
    return 0;
}
```

Sample input: `4 9 / 2 7 11 15` **Sample output:** `0 1`

Answer: Output: 0 1

Example 2. Count subarrays whose sum equals K.

Solution: Keep a map of prefix-sum counts, starting with {0: 1}.

```python
n, k = map(int, input().split())
a = list(map(int, input().split()))
cnt = {0: 1}
pre = ans = 0
for x in a:
    pre += x
    ans += cnt.get(pre - k, 0)
    cnt[pre] = cnt.get(pre, 0) + 1
print(ans)
```

Sample input: `5 3 / 1 2 1 2 1` **Sample output:** `4`

Answer: Output: 4

Example 3. Print the value that appears more than n/2 times, or -1 if there is none.

Solution: Count with a dictionary and test the maximum count.

```python
from collections import Counter
n = int(input())
a = list(map(int, input().split()))
v, c = Counter(a).most_common(1)[0]
print(v if c > n // 2 else -1)
```

Sample input: `7 / 3 3 4 2 3 3 1` **Sample output:** `3`

Answer: Output: 3

Example 4. Print the length of the longest run of consecutive integers (in any order) in the array.

Solution: Use a set; start counting only from numbers whose predecessor is missing.

```python
n = int(input())
s = set(map(int, input().split()))
best = 0
for x in s:
    if x - 1 not in s:
        y = x
        while y + 1 in s:
            y += 1
        best = max(best, y - x + 1)
print(best)
```

Sample input: `6 / 100 4 200 1 3 2` **Sample output:** `4`

Answer: Output: 4

Shortcuts

- Counter makes frequency questions one-liners.
- For 'at most once' conditions use a set of seen values.
- Use a prefix-sum map for any 'subarray with sum' question.

Common mistakes

- Updating the map before checking, so an element pairs with itself.
- Forgetting the initial {0: 1} in prefix-sum counting.
- Using unordered containers when sorted order is needed.

Practice questions: Hashing, Sets and Maps

1. Print the first element that appears twice (in order of the second occurrence), or -1. *Sample input:* `6 / 3 5 4 5 3 7` **Basic**
2. Print YES if two strings are anagrams. *Sample input:* `triangle / integral` **Basic**
3. Print the number of distinct elements. *Sample input:* `7 / 1 2 2 3 3 3 4` **Basic**
4. Print the element with the highest frequency (smallest if tied). *Sample input:* `8 / 4 4 5 5 6 6 6 4` **Basic**
5. Print the number of pairs (i < j) with a[i] == a[j]. *Sample input:* `6 / 1 2 1 2 1 3` **Medium**
6. Print the number of subarrays with sum 0. *Sample input:* `6 / 1 -1 2 -2 3 -3` **Medium**
7. Print the length of the longest consecutive-integer run. *Sample input:* `7 / 9 1 8 2 3 7 10` **Medium**
8. Group anagrams; print the number of groups. *Sample input:* `6 / eat tea tan ate nat bat` **Medium**

Answers: see the answer key at the end of this guide.

08 ADVANCED CODING · IMPORTANT
Two Pointers and Sliding Window

These two patterns replace nested loops with a pair of moving indices. They solve many 'pair', 'subarray' and 'longest substring' problems in O(n).

Start from zero

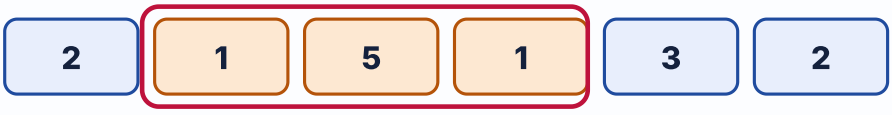
Two pointers on a sorted array

Place one pointer at the start (i) and one at the end (j). If a[i] + a[j] is too small, move i right; if too large, move j left. This finds a pair with a given sum in O(n) after sorting.

Same-direction pointers

A slow pointer marks the next free spot and a fast pointer scans. Removing duplicates from a sorted array in place uses this pattern: the fast pointer finds a new value and the slow pointer stores it.

Sliding window of size 3: add the new element, remove the old one



Window sums: 8, 7, 9, 6 → maximum 9. New sum = old sum + entering − leaving.

Brute force recomputes each window: O(n·k). Sliding window: O(n).

A sliding window of size 3

Fixed-size window

For the maximum sum of k consecutive elements, compute the first window's sum, then slide: add the incoming element and subtract the outgoing element. Total O(n).

Variable-size window

Expand the right end, and shrink the left end while the window breaks the rule. Used for the longest substring without repeats, the smallest subarray with sum ≥ S and the longest subarray with at most k distinct values.

When it works

Sliding window needs a 'monotone' rule: adding elements only makes the property harder (or easier), so you can move the left end only forward. For arrays with negative numbers and a sum target, use a prefix-sum map instead.

Formula and rule box

- Pair sum on sorted array: i from the left, j from the right
- Window sum: new = old + a[right] − a[left]
- Longest unique substring: move left past the last duplicate
- Time: O(n) because each pointer moves at most n times
- Smallest subarray with sum ≥ S: shrink while sum ≥ S

Step-by-step method

1. Check if the array is sorted (or can be sorted).

2. Pick the pattern: opposite ends, slow–fast or window.
3. Define what moves the left and right pointers.
4. Update the answer at each step.

Solved examples

Example 1. Print YES if a sorted array contains two numbers that add to target.

Solution: Use pointers at both ends and move them according to the sum.

```python
n, target = map(int, input().split())
a = list(map(int, input().split()))
i, j, ok = 0, n - 1, False
while i < j:
    s = a[i] + a[j]
    if s == target:
        ok = True
        break
    if s < target:
        i += 1
    else:
        j -= 1
print('YES' if ok else 'NO')
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int n, target; cin >> n >> target;
    int a[100000];
    for (int i = 0; i < n; i++) cin >> a[i];
    int i = 0, j = n - 1; bool ok = false;
    while (i < j) {
        int s = a[i] + a[j];
        if (s == target) { ok = true; break; }
        if (s < target) i++; else j--;
    }
    cout << (ok ? "YES" : "NO") << endl;
    return 0;
}
```

Sample input: `6 14 / 1 3 5 7 9 11` **Sample output:** `YES`

Answer: Output: YES

Example 2. Print the maximum sum of k consecutive elements.

Solution: Compute the first window, then slide it by adding the new element and removing the old one.

```python
n, k = map(int, input().split())
a = list(map(int, input().split()))
cur = sum(a[:k])
best = cur
for i in range(k, n):
    cur += a[i] - a[i - k]
    best = max(best, cur)
print(best)
```

Sample input: `6 3 / 2 1 5 1 3 2` **Sample output:** `9`

Answer: Output: 9

Example 3. Print the length of the longest substring without repeating characters.

Solution: Keep the last index of each character. When a repeat appears inside the window, move the left end past it.

```python
s = input().strip()
last = {}
left = best = 0
for i, c in enumerate(s):
    if c in last and last[c] >= left:
        left = last[c] + 1
    last[c] = i
    best = max(best, i - left + 1)
print(best)
```

Sample input: `abcabcbb` **Sample output:** `3`

Answer: Output: 3

Example 4. Print the length of the smallest subarray whose sum is at least S (positive numbers), or 0.

Solution: Expand right, and while the sum is at least S, record the length and shrink from the left.

```python
n, S = map(int, input().split())
a = list(map(int, input().split()))
left = cur = 0
best = n + 1
for right in range(n):
    cur += a[right]
    while cur >= S:
        best = min(best, right - left + 1)
        cur -= a[left]
        left += 1
print(best if best <= n else 0)
```

Sample input: `6 7 / 2 3 1 2 4 3` **Sample output:** `2`

Answer: Output: 2

Shortcuts

- Sort first if the order of the array does not matter.
- In a window, each element enters once and leaves once.
- Use two pointers when the answer needs a pair; use a window for contiguous ranges.

Common mistakes

- Using sliding window when the array has negative numbers and a sum target.
- Not moving the left pointer past the previous duplicate.
- Off-by-one in the window length (right − left + 1).

Practice questions: Two Pointers and Sliding Window

1. Print the length of the array after removing duplicates from a sorted array. *Sample input:* `7 / 1 1 2 2 3 4 4` **Basic**
2. Print the maximum sum of 2 consecutive elements. *Sample input:* `5 / 1 4 2 10 2` **Basic**
3. Print the number of pairs with sum 10 in a sorted array. *Sample input:* `6 / 1 3 5 5 7 9` **Basic**
4. Print the length of the longest substring with at most 2 distinct characters. *Sample input:* `eceba` **Basic**
5. Print YES if the string has two pointers meeting as a palindrome. *Sample input:* `level` **Medium**
6. Print the maximum number of consecutive 1s in a binary array. *Sample input:* `8 / 1 1 0 1 1 1 0 1` **Medium**
7. Print the smallest window length with sum at least 11. *Sample input:* `6 / 1 2 3 4 5 6` **Medium**
8. Print the length of the longest subarray with sum at most 8 (positive numbers). *Sample input:* `6 / 1 2 3 4 1 1` **Medium**

Answers: see the answer key at the end of this guide.

09 ADVANCED CODING · IMPORTANT

Recursion and Backtracking

Recursion solves a problem by solving a smaller copy of itself. Backtracking adds the idea of trying a choice, exploring, and undoing the choice. Subsets, permutations and puzzles all follow this pattern.

Start from zero

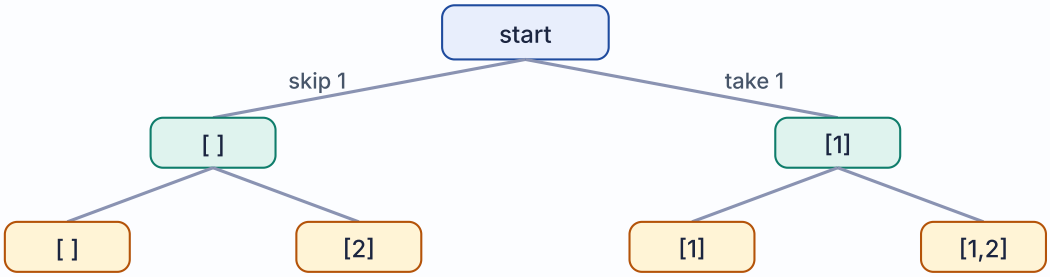
The two parts of recursion

Every recursive function needs a **base case** that stops the recursion and a **recursive case** that calls itself with a smaller input. Factorial: n! = n × (n − 1)!, with base case 0! = 1.

The call stack

Each call waits for the one it started. Very deep recursion (more than about 1000 levels in Python) can crash; raise the limit with `sys.setrecursionlimit` or convert to a loop.

Subsets of [1, 2, 3]: include or skip each element



After 3 decisions: $2^3 = 8$ subsets.

Backtracking = make a choice, solve the smaller problem, undo the choice.

A recursion tree for subsets

Memoisation

Plain Fibonacci repeats the same subproblems and takes exponential time. Store results in a dictionary or use `functools.lru_cache` so each subproblem is solved once. This is the bridge to dynamic programming.

Generating subsets and permutations

Subsets: for each element decide include or skip, giving 2^n results. Permutations: choose an unused element, recurse, then unchoose it (backtrack). Python has `itertools`, but understanding the recursion matters for puzzles.

N-Queens and constraint puzzles

Place one queen per row. For each column, check that no earlier queen attacks it (same column or diagonal). If the placement works, recurse to the next row; if it fails, try the next column. Backtracking prunes impossible branches early.

Other classics

Tower of Hanoi needs $2^n - 1$ moves. Combination sum picks numbers to reach a target, allowing repeats. Maze problems explore four directions and mark cells visited.

Formula and rule box

- Base case first, then a smaller recursive call
- Fibonacci with memo: O(n)
- Subsets: 2^n; permutations: n!
- Tower of Hanoi moves: $2^n - 1$
- Backtracking: choose → explore → unchoose
- N-Queens (n = 4): 2 solutions; (n = 8): 92

Step-by-step method

1. Define the smaller subproblem.
2. Write the base case.
3. Make the recursive call and combine the answer.
4. For backtracking, undo the change after returning.

Solved examples

Example 1. Print the sum of the digits of n using recursion.

Solution: The sum of digits of n is the last digit plus the sum for n // 10; the base case is n = 0.

```Python
def f(n):
    if n == 0:
        return 0
    return n % 10 + f(n // 10)
print(f(int(input())))
```

```C++
#include <iostream>
using namespace std;
int f(long long n) { return n == 0 ? 0 : n % 10 + f(n / 10); }
int main() {
    long long n; cin >> n;
    cout << f(n) << endl;
    return 0;
}
```

Sample input: 4825 **Sample output:** 19

Answer: Output: 19

Example 2. Print all subsets of the first n numbers 1..n, each on its own line in the order generated by include-first recursion. Print 'empty' for the empty set.

Solution: At each element choose to include it or skip it.

```python
n = int(input())
res = []
def go(i, cur):
    if i > n:
        res.append(cur[:])
        return
    cur.append(i)
    go(i + 1, cur)
    cur.pop()
    go(i + 1, cur)
go(1, [])
for r in res:
    print(*r if r else ['empty'])
```

Sample input: 3 **Sample output:** 1 2 3 / 1 2 / 1 3 / 1 / 2 3 / 2 / 3 / empty

Answer: Output: 1 2 3 / 1 2 / 1 3 / 1 / 2 3 / 2 / 3 / empty

Example 3. Print the number of ways to place n queens on an n × n board so that none attack each other.

Solution: Place queens row by row and keep sets of used columns and diagonals.

```python
n = int(input())
cols, d1, d2 = set(), set(), set()
def go(r):
    if r == n:
        return 1
    c = 0
    for col in range(n):
        if col in cols or r - col in d1 or r + col in d2:
            continue
        cols.add(col); d1.add(r - col); d2.add(r + col)
        c += go(r + 1)
        cols.remove(col); d1.remove(r - col); d2.remove(r + col)
    return c
print(go(0))
```

Sample input: 6 **Sample output:** 4

Answer: Output: 4

Example 4. Print the number of ways to climb n stairs taking 1 or 2 steps at a time, using memoised recursion.

Solution: ways(n) = ways(n − 1) + ways(n − 2). Memoisation avoids repeating work.

```python
from functools import lru_cache
@lru_cache(None)
def ways(n):
    if n <= 1:
        return 1
    return ways(n - 1) + ways(n - 2)
print(ways(int(input())))
```

Sample input: `10` **Sample output:** `89`

Answer: Output: 89

Shortcuts

- Write the base case first.
- lru_cache turns slow recursion into fast DP in one line.
- For backtracking remember to undo every change.

Common mistakes

- Missing base case causing infinite recursion.
- Forgetting to unchoose after the recursive call.
- Using plain recursion for Fibonacci with n above 35.

Practice questions: Recursion and Backtracking

1. Print n! using recursion. *Sample input:* 6 **Basic**
2. Print the nth Fibonacci number (F(1) = F(2) = 1) with memoisation. *Sample input:* `20` **Basic**
3. Print the number of subsets of an n-element set. *Sample input:* 5 **Basic**
4. Print the number of moves in the Tower of Hanoi with n disks. *Sample input:* 6 **Basic**
5. Print the number of permutations of the string (all letters distinct). *Sample input:* `abcd` **Medium**
6. Print the number of N-Queens solutions for n = 5. *Sample input:* 5 **Medium**
7. Print the number of subsets of [2, 3, 5, 7] with sum 10. *Sample input:* `4 / 2 3 5 7` **Medium**
8. Print the number of paths from top-left to bottom-right of an r × c grid moving right or down. *Sample input:* `3 4` **Medium**

Answers: see the answer key at the end of this guide.

10 ADVANCED CODING · IMPORTANT
Stack and Queue

A stack is last-in, first-out and a queue is first-in, first-out. They model undo operations, nested brackets, expression evaluation and level-by-level exploration.

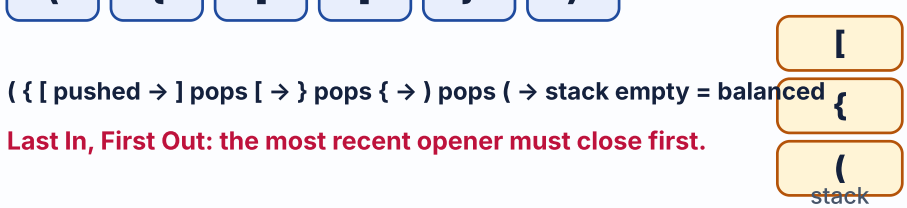
Start from zero

Stack

Operations: push (add on top), pop (remove top), peek (look at top). In Python use a list with append and pop. Typical uses: balanced brackets, reversing, undo, evaluating postfix expressions, next-greater-element.

Queue and deque

Operations: enqueue at the back, dequeue from the front. In Python use `collections.deque` (popleft is O(1), unlike list.pop(0)). Typical uses: BFS, scheduling, sliding-window maximum with a deque.



A stack checking brackets

Balanced brackets

Scan the string. Push every opening bracket. For each closing bracket, the stack must not be empty and its top must be the matching opener; pop it. The string is balanced if the stack is empty at the end.

Next greater element

Process from left to right with a stack of indices whose answer is not found. When a bigger element arrives, pop smaller ones and set their answer. Each index is pushed and popped once: O(n).

Postfix evaluation

Read tokens: push numbers; for an operator pop two numbers, apply the operator (second popped is the left operand) and push the result. The final stack top is the answer.

Monotonic stack idea

Keep the stack in increasing or decreasing order. Used for nearest smaller or greater values, daily temperatures and histogram areas.

Formula and rule box

- Stack: LIFO; queue: FIFO
- Python stack: list.append / list.pop
- Python queue: collections.deque, popleft()
- Balanced brackets ⇔ stack empty at the end
- Next greater element: O(n) with a monotonic stack
- Postfix: operand order matters for − and ÷

Step-by-step method

1. Decide: do you need LIFO or FIFO?
2. Define what you push and when you pop.
3. Handle the empty stack carefully.
4. Check the final state of the stack.

Solved examples

Example 1. Print YES if the bracket string is balanced, else NO.

Solution: Push openers; on a closer, compare with the top of the stack.

```python
s = input().strip()
pair = {')': '(', ']': '[', '}': '{'}
st = []
ok = True
for c in s:
    if c in '([{':
        st.append(c)
    elif c in pair:
        if not st or st.pop() != pair[c]:
            ok = False
            break
print('YES' if ok and not st else 'NO')
```

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;
int main() {
    string s; cin >> s;
    stack<char> st; bool ok = true;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) { ok = false; break; }
            char t = st.top(); st.pop();
            if ((c == ')' && t != '(') || (c == ']' && t != '[') || (c == '}' && t !=
'{')) { ok = false; break; }
        }
    }
    cout << ((ok && st.empty()) ? "YES" : "NO") << endl;
    return 0;
}
```

Sample input: `{[()]}` **Sample output:** `YES`

Answer: Output: YES

Example 2. For each element print the next greater element to its right, or -1.

Solution: Use a stack of indices waiting for a greater value.

```
Python
n = int(input())
a = list(map(int, input().split()))
res = [-1] * n
st = []
for i, x in enumerate(a):
    while st and a[st[-1]] < x:
        res[st.pop()] = x
    st.append(i)
print(*res)
```

Sample input: `5 / 4 5 2 25 7` **Sample output:** `5 25 25 -1 -1`

Answer: Output: 5 25 25 -1 -1

Example 3. Evaluate a postfix expression with single-digit operands, e.g. 231*+9-.

Solution: Push digits; on an operator pop two numbers and push the result.

```
Python
s = input().strip()
st = []
for c in s:
    if c.isdigit():
        st.append(int(c))
    else:
        b = st.pop(); a = st.pop()
        st.append(a + b if c == '+' else a - b if c == '-' else a * b if c == '*' else a
// b)
print(st[-1])
```

Sample input: `231*+9-` **Sample output:** `-4`

Answer: Output: -4

Example 4. Reverse the order of words in a sentence using a stack.

Solution: Push each word and pop them to get the reverse order.

```
Python
words = input().split()
st = []
for w in words:
    st.append(w)
out = []
while st:
    out.append(st.pop())
print(' '.join(out))
```

Sample input: `TCS NQT is fun` **Sample output:** `fun is NQT TCS`

Answer: Output: fun is NQT TCS

Shortcuts

- Use deque for queues in Python.
- Push indices, not values, when you need positions.
- Check 'not stack' before popping.

Common mistakes

- Popping from an empty stack.
- Using list.pop(0) as a queue.
- Swapping operand order for − and ÷.

Practice questions: Stack and Queue

1. Print the length of the longest valid parentheses substring. *Sample input:* `(()())` **Basic**
2. Print YES if the brackets are balanced. *Sample input:* `[()]` **Basic**
3. For each day print how many days until a warmer temperature (0 if none). *Sample input:* `6 / 73 74 75 71 69 72` **Basic**
4. Evaluate the postfix expression. *Sample input:* `53+82-*` **Basic**
5. Simulate: push 1,2,3 then pop twice, then push 4. Print the stack from bottom to top. *Sample input:* `1 2 3` **Medium**
6. Print the maximum element in each window of size 3. *Sample input:* `6 / 1 3 -1 -3 5 3` **Medium**
7. Print the reverse of the string using a stack. *Sample input:* `stack` **Medium**
8. Print the number of valid bracket pairs removed to balance: minimum removals for '())('. *Sample input:* `())(` **Medium**

Answers: see the answer key at the end of this guide.

11 ADVANCED CODING · IMPORTANT
Matrix and 2D Arrays

A matrix is an array of arrays. Matrix problems are mostly about traversal order: rows, columns, diagonals, spirals and rotations. Careful index handling is the whole skill.

Start from zero

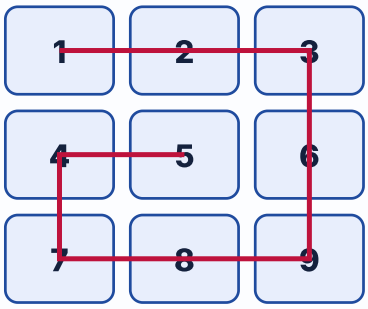
Reading a matrix

First line: rows r and columns c. Then r lines of c numbers. Python: `g = [list(map(int, input().split())) for _ in range(r)]`. C++: a 2D vector or array.

Traversals

Row-wise: two nested loops. Column-wise: swap the loops. Main diagonal: cells where i == j. Anti-diagonal: i + j == n − 1. Boundary: cells in the first or last row or column.

Spiral order of a 3×3 matrix: 1 2 3 6 9 8 7 4 5



Transpose: A[i][j] ↔ A[j][i]

Rotate 90° clockwise:
transpose, then reverse each row

Diagonal: i == j Anti-diagonal: i + j == n − 1

Spiral order, transpose and diagonals

Transpose and rotation

Transpose swaps A[i][j] with A[j][i]. To rotate 90° clockwise, transpose and then reverse each row. To rotate anticlockwise, transpose and then reverse the order of rows.

Spiral order

Keep four boundaries: top, bottom, left, right. Print the top row, then the right column, then the bottom row (reversed), then the left column (reversed), shrinking the boundaries after each side. Stop when they cross.

Row and column sums, searching

Compute sums with one pass. In a matrix sorted by rows and columns, start from the top-right corner: move left if the value is too large, down if too small. O(r + c).

Grid neighbours

Use direction arrays: `dirs = [(1,0),(-1,0),(0,1),(0,-1)]` and check bounds before accessing a neighbour. This is the base of flood fill and path problems.

Formula and rule box

- Main diagonal: i == j; anti-diagonal: i + j == n − 1
- Rotate 90° clockwise = transpose + reverse each row
- Four neighbours: (±1, 0), (0, ±1)
- Spiral: top → right → bottom → left, shrink boundaries
- Sorted matrix search: start at the top-right corner

Step-by-step method

1. Draw a small matrix and mark the index pattern.
2. Write the loop limits for each side or direction.
3. Check bounds before using neighbours.
4. Test with 1 × 1, 1 × n and n × 1 matrices.

Solved examples

Example 1. Print the matrix in spiral order.

Solution: Peel the outer layer by shrinking four boundaries.

```python
r, c = map(int, input().split())
g = [list(map(int, input().split())) for _ in range(r)]
res = []
top, bot, left, right = 0, r - 1, 0, c - 1
while top <= bot and left <= right:
    for j in range(left, right + 1): res.append(g[top][j])
    top += 1
    for i in range(top, bot + 1): res.append(g[i][right])
    right -= 1
    if top <= bot:
        for j in range(right, left - 1, -1): res.append(g[bot][j])
        bot -= 1
    if left <= right:
        for i in range(bot, top - 1, -1): res.append(g[i][left])
        left += 1
print(*res)
```

Sample input: `3 3 / 1 2 3 / 4 5 6 / 7 8 9` **Sample output:** `1 2 3 6 9 8 7 4 5`

Answer: Output: 1 2 3 6 9 8 7 4 5

Example 2. Rotate a square matrix 90° clockwise and print it.

Solution: Transpose, then reverse each row.

```python
n = int(input())
g = [list(map(int, input().split())) for _ in range(n)]
t = [list(r) for r in zip(*g)]
for row in t:
    print(*row[::-1])
```

Sample input: `3 / 1 2 3 / 4 5 6 / 7 8 9` **Sample output:** `7 4 1 / 8 5 2 / 9 6 3`

Answer: Output: 7 4 1 / 8 5 2 / 9 6 3

Example 3. Print the sums of the main diagonal and the anti-diagonal.

Solution: Use i == j and i + j == n − 1.

```
Python
n = int(input())
g = [list(map(int, input().split())) for _ in range(n)]
d1 = sum(g[i][i] for i in range(n))
d2 = sum(g[i][n - 1 - i] for i in range(n))
print(d1, d2)
```

```
C++
#include <iostream>
using namespace std;
int main() {
    int n; cin >> n;
    int g[100][100]; long long d1 = 0, d2 = 0;
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) cin >> g[i][j];
    for (int i = 0; i < n; i++) { d1 += g[i][i]; d2 += g[i][n - 1 - i]; }
    cout << d1 << " " << d2 << endl;
    return 0;
}
```

Sample input: `3 / 1 2 3 / 4 5 6 / 7 8 9` **Sample output:** `15 15`

Answer: Output: 15 15

Example 4. Search for x in a matrix whose rows and columns are sorted. Print YES or NO.

Solution: Start at the top-right corner and move left or down.

```
Python
r, c, x = map(int, input().split())
g = [list(map(int, input().split())) for _ in range(r)]
i, j, ok = 0, c - 1, False
while i < r and j >= 0:
    if g[i][j] == x:
        ok = True
        break
    if g[i][j] > x:
        j -= 1
    else:
        i += 1
print('YES' if ok else 'NO')
```

Sample input: `3 3 5 / 1 4 7 / 2 5 8 / 3 6 9` **Sample output:** YES

Answer: Output: YES

Shortcuts

- zip(*g) transposes a matrix in Python.
- Use direction arrays instead of four copies of code.
- Test on a 1 × n matrix.

Common mistakes

- Swapping rows and columns in loops.
- Not checking bounds before reading a neighbour.
- Printing the spiral twice for the last row or column.

Practice questions: Matrix and 2D Arrays

1. Print the sum of all elements of the matrix. *Sample input:* `2 3 / 1 2 3 / 4 5 6` **Basic**
2. Print the transpose of the matrix (one row per line, separated by ' / '). *Sample input:* `2 3 / 1 2 3 / 4 5 6` **Basic**
3. Print the sum of the boundary elements of the matrix. *Sample input:* `3 3 / 1 2 3 / 4 5 6 / 7 8 9` **Basic**
4. Print the maximum row sum. *Sample input:* `3 3 / 1 2 3 / 4 5 6 / 7 8 9` **Basic**
5. Print YES if the square matrix is symmetric. *Sample input:* `3 / 1 2 3 / 2 4 5 / 3 5 6` **Medium**
6. Print the sum of the elements above the main diagonal. *Sample input:* `3 / 1 2 3 / 4 5 6 / 7 8 9` **Medium**
7. Print the number of islands (groups of connected 1s, 4-direction) in the grid. *Sample input:* `3 4 / 1 1 0 0 / 0 1 0 1 / 1 0 0 1` **Medium**
8. Print the minimum path sum from top-left to bottom-right moving right or down. *Sample input:* `3 3 / 1 3 1 / 1 5 1 / 4 2 1` **Medium**

Answers: see the answer key at the end of this guide.

12 ADVANCED CODING · IMPORTANT
Greedy Algorithms

A greedy algorithm makes the locally best choice at each step and never looks back. It is fast and simple, but it works only when the problem has a special structure, so you must understand why it is correct.

Start from zero

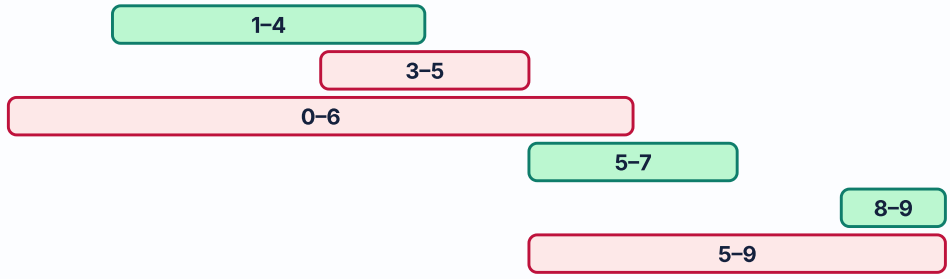
The greedy idea

At each step take the choice that looks best right now (earliest finish, largest coin, highest ratio). If the problem has the greedy-choice property, these local choices add up to a global optimum.

Activity selection

Given intervals, pick the maximum number that do not overlap. Sort by end time; take an interval if its start is at least the end of the last chosen one. This is provably optimal.

Activity selection: always pick the activity that ends earliest



Chosen: (1,4), (5,7), (8,9) → 3 activities. Sort by end time, then pick if start ≥ last end.

Pick the activity that ends first

Coins and change

With coin systems like 1, 5, 10, 25, taking the largest coin first gives the minimum coins. For arbitrary coins (for example 1, 3, 4), greedy can fail: 6 = 4 + 1 + 1 (3 coins) but 3 + 3 uses 2 coins, so use dynamic programming.

Fractional knapsack

Sort items by value per weight and take as much as possible of the best items. Greedy is optimal when items can be split (unlike 0/1 knapsack).

Sorting-based greedy

Many greedy problems start with a sort: minimum platforms (sort arrivals and departures), meeting rooms, assigning cookies to children, job scheduling with deadlines, jump game (track the farthest reachable index).

How to check a greedy idea

Try to find a counterexample with small numbers. If you cannot break it and an exchange argument works (swapping a non-greedy choice for the greedy one never hurts), the greedy is probably right.

Formula and rule box

- Activity selection: sort by end time
- Fractional knapsack: sort by value/weight
- Greedy coin change works only for canonical systems
- Jump game: reach = max(reach, i + a[i])
- Minimum platforms: sort arrivals and departures

Step-by-step method

1. Decide what 'best choice' means.
2. Sort by the right key.
3. Take choices in order while they remain feasible.
4. Try a small counterexample to test the idea.

Solved examples

Example 1. Print the maximum number of non-overlapping activities. Each line has start and end.

Solution: Sort by end time and take an activity if it starts after the previous one ended.

```python
n = int(input())
acts = [tuple(map(int, input().split())) for _ in range(n)]
acts.sort(key=lambda x: x[1])
cnt, last = 0, -1
for s, e in acts:
    if s >= last:
        cnt += 1
        last = e
print(cnt)
```

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int main() {
    int n; cin >> n;
    vector<pair<int,int>> v(n);
    for (auto &p : v) cin >> p.second >> p.first;
    sort(v.begin(), v.end());
    int cnt = 0, last = -1;
    for (auto &p : v) if (p.second >= last) { cnt++; last = p.first; }
    cout << cnt << endl;
    return 0;
}
```

Sample input: `6 / 1 4 / 3 5 / 0 6 / 5 7 / 8 9 / 5 9` **Sample output:** `3`

Answer: Output: 3

Example 2. Print the minimum number of coins to make amount A with coins 1, 5, 10, 25 (canonical system).

Solution: Take as many of the largest coin as possible, then move to the next.

```python
a = int(input())
cnt = 0
for c in (25, 10, 5, 1):
    cnt += a // c
    a %= c
print(cnt)
```

Sample input: 93 **Sample output:** 8

Answer: Output: 8

Example 3. Print YES if you can reach the last index of the array, where a[i] is the maximum jump length from i.

Solution: Track the farthest index you can reach as you scan.

```python
n = int(input())
a = list(map(int, input().split()))
reach = 0
ok = True
for i in range(n):
    if i > reach:
        ok = False
        break
    reach = max(reach, i + a[i])
print('YES' if ok else 'NO')
```

Sample input: 5 / 2 3 1 1 4 **Sample output:** YES

Answer: Output: YES

Example 4. Print the minimum number of platforms needed so that no train waits. Input: n, arrivals, departures (HHMM as integers).

Solution: Sort arrivals and departures separately. Sweep through time and count trains present at the same time.

```python
n = int(input())
arr = sorted(map(int, input().split()))
dep = sorted(map(int, input().split()))
i = j = cur = best = 0
while i < n:
    if arr[i] <= dep[j]:
        cur += 1
        best = max(best, cur)
        i += 1
    else:
        cur -= 1
        j += 1
print(best)
```

Sample input: `6 / 900 940 950 1100 1500 1800 / 910 1200 1120 1130 1900 2000` **Sample output:** `3`

Answer: Output: 3

Shortcuts

- Sort by end time for scheduling, by ratio for knapsack.
- Always test a greedy idea on a tiny counterexample.
- If greedy fails, think dynamic programming.

Common mistakes

- Using greedy on non-canonical coin systems.
- Sorting by start time instead of end time.
- Not checking that a greedy is correct.

Practice questions: Greedy Algorithms

1. Print the minimum number of coins for amount 67 using 1, 2, 5, 10, 20, 50. *Sample input:* 67 — Basic
2. Print the maximum number of meetings in one room (start end per line). *Sample input:* `3 / 1 2 / 3 4 / 2 3` — Basic
3. Print the maximum number of children satisfied. Greed factors, then cookie sizes. *Sample input:* `3 / 1 2 3 / 2 / 1 1` — Basic
4. Print the smallest number of jumps to reach the end, where a[i] is the max jump. *Sample input:* `6 / 2 3 1 1 2 4` — Basic
5. Print the maximum total value in the fractional knapsack: capacity 50, items (value, weight): (60,10),(100,20),(120,30). *Sample input:* `50` — Medium
6. Print the minimum sum of absolute differences after pairing sorted arrays: [4,1,8,7] and [2,3,6,5]. *Sample input:* `4 / 4 1 8 7 / 2 3 6 5` — Medium
7. Print the maximum profit from one buy and one sell: prices 7 1 5 3 6 4. *Sample input:* `6 / 7 1 5 3 6 4` — Medium
8. Print the lexicographically smallest number made by removing 1 digit from 1432219. *Sample input:* 1432219 — Medium

Answers: see the answer key at the end of this guide.

13 ADVANCED CODING · MUST-DO
Dynamic Programming

Dynamic programming (DP) solves a problem by building the answer from answers to smaller overlapping subproblems and remembering them. DP is the usual tool for the Medium coding problem, so it is worth learning well.

Start from zero

When to use DP

Use DP when (1) the problem breaks into smaller versions of itself and (2) the same subproblems repeat. Typical phrases: minimum or maximum, number of ways, can we make, longest. If a greedy idea fails and brute force is exponential, try DP.

The five steps

1. Define the state (what dp[i] or dp[i][j] means). 2. Write the transition (how it depends on smaller states). 3. Set the base cases. 4. Choose the order (usually small to large). 5. Read the answer from the right cell.

Coin change DP (coins 1, 3, 4; minimum coins for each amount 0 to 6)



dp[a] = 1 + min(dp[a − c]) over coins c ≤ a. dp[6] = 1 + min(dp[5], dp[3], dp[2]) = 1 + 1 = 2 (3 + 3)

Greedy (take the largest coin) would give 4 + 1 + 1 = 3 coins, so DP is needed here.

A DP table for minimum coins

1D examples

Climbing stairs: dp[i] = dp[i−1] + dp[i−2]. House robber: dp[i] = max(dp[i−1], dp[i−2] + a[i]). Minimum coins: dp[a] = 1 + min(dp[a − c]) over coins c ≤ a, with dp[0] = 0.

2D examples

Longest common subsequence: if the characters match, dp[i][j] = dp[i−1][j−1] + 1; otherwise max(dp[i−1][j], dp[i][j−1]). Grid paths: dp[i][j] = dp[i−1][j] + dp[i][j−1]. 0/1 knapsack: dp[w] = max(dp[w], dp[w − wt] + value), looping w from high to low.

Longest increasing subsequence

dp[i] = 1 + max(dp[j]) for j < i with a[j] < a[i]. This is $O(n^2)$; the answer is the maximum dp[i]. A faster O(n log n) method exists with binary search.

Memoisation versus table

Top-down (recursion with memo) is easier to write; bottom-up (loops) is faster and avoids recursion limits. Both give the same answer.

Formula and rule box

- Climbing stairs: dp[n] = dp[n−1] + dp[n−2]
- Coin change: dp[a] = 1 + min(dp[a − c])
- LCS: match → dp[i−1][j−1] + 1, else max of neighbours
- 0/1 knapsack: dp[w] = max(dp[w], dp[w − wt] + val)
- LIS: dp[i] = 1 + max dp[j] with a[j] < a[i]
- Grid paths: dp[i][j] = dp[i−1][j] + dp[i][j−1]

Step-by-step method

1. Define the state in one sentence.
2. Write the transition and base cases.
3. Fill the table in the right order.
4. Print the cell that holds the answer.

Solved examples

Example 1. Print the minimum number of coins needed to make amount A from the given coin values, or -1.

Solution: dp[a] is the minimum coins for amount a. Try each coin.

```python
n, A = map(int, input().split())
coins = list(map(int, input().split()))
INF = 10**9
dp = [0] + [INF] * A
for a in range(1, A + 1):
    for c in coins:
        if c <= a and dp[a - c] + 1 < dp[a]:
            dp[a] = dp[a - c] + 1
print(dp[A] if dp[A] < INF else -1)
```

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, A; cin >> n >> A;
    vector<int> c(n);
    for (auto &x : c) cin >> x;
    const int INF = 1e9;
    vector<int> dp(A + 1, INF); dp[0] = 0;
    for (int a = 1; a <= A; a++)
        for (int x : c)
            if (x <= a && dp[a - x] + 1 < dp[a]) dp[a] = dp[a - x] + 1;
    cout << (dp[A] < INF ? dp[A] : -1) << endl;
    return 0;
}
```

Sample input: `3 6 / 1 3 4` **Sample output:** `2`

Answer: Output: 2

Example 2. Print the length of the longest common subsequence of two strings.

Solution: dp[i][j] is the LCS of the first i and first j characters.

```python
a = input().strip()
b = input().strip()
dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
for i in range(1, len(a) + 1):
    for j in range(1, len(b) + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
print(dp[len(a)][len(b)])
```

Sample input: `ABCBDAB / BDCABA` **Sample output:** `4`

Answer: Output: 4

Example 3. 0/1 knapsack: print the maximum value for capacity W with n items given as weight and value.

Solution: Loop weights from high to low so each item is used at most once.

```python
n, W = map(int, input().split())
items = [tuple(map(int, input().split())) for _ in range(n)]
dp = [0] * (W + 1)
for wt, val in items:
    for w in range(W, wt - 1, -1):
        dp[w] = max(dp[w], dp[w - wt] + val)
print(dp[W])
```

Sample input: `3 50 / 10 60 / 20 100 / 30 120` **Sample output:** `220`

Answer: Output: 220

Example 4. Print the length of the longest increasing subsequence.

Solution: dp[i] is the length of the best subsequence ending at i.

```python
n = int(input())
a = list(map(int, input().split()))
dp = [1] * n
for i in range(n):
    for j in range(i):
        if a[j] < a[i]:
            dp[i] = max(dp[i], dp[j] + 1)
print(max(dp))
```

Sample input: `8 / 10 9 2 5 3 7 101 18` **Sample output:** `4`

Answer: Output: 4

Shortcuts

- State first, transition second.
- If recursion repeats subproblems, add memoisation.
- Use 1D arrays when only the previous row is needed.

Common mistakes

- Looping knapsack weights upwards (item is reused).
- Wrong base case for dp[0].
- Mixing 0-based and 1-based indices in LCS.

Practice questions: Dynamic Programming

1. Print the number of ways to climb n stairs with steps of 1 or 2. *Sample input:* `10` **Basic**
2. Print the maximum money from non-adjacent houses (house robber). *Sample input:* `5 / 2 7 9 3 1` **Basic**
3. Print the number of paths from top-left to bottom-right in a 3 × 4 grid (right/down moves). *Sample input:* `3 4` **Basic**
4. Print YES if a subset of the numbers sums to the target. *Sample input:* `5 9 / 3 34 4 12 5` **Basic**
5. Print the minimum number of coins for 11 with coins 1, 2, 5. *Sample input:* `3 11 / 1 2 5` **Medium**
6. Print the length of the longest increasing subsequence. *Sample input:* `6 / 5 8 3 7 9 1` **Medium**
7. Print the edit distance between 'kitten' and 'sitting'. *Sample input:* `kitten / sitting` **Medium**
8. Print the number of ways to make amount 5 using coins 1, 2, 5 (order does not matter). *Sample input:* `5` **Medium**

Answers: see the answer key at the end of this guide.

14 ADVANCED CODING · GOOD TO KNOW
Linked Lists, Trees and Graphs

These structures appear less often in the Easy and Medium problems but are worth knowing at a basic level: reversing a list, tree traversals, and breadth-first or depth-first search on graphs and grids.

Start from zero

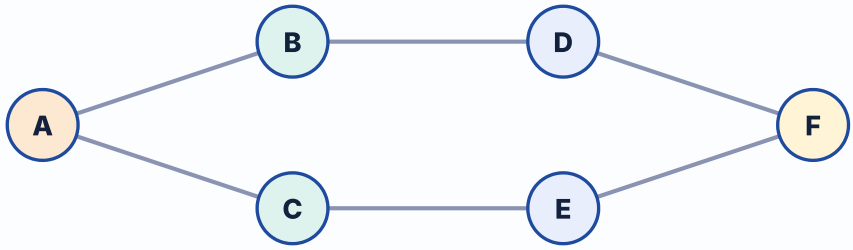
Linked list

A chain of nodes, each holding a value and a pointer to the next node. Typical tasks: reverse the list (keep previous, current, next), find the middle (slow and fast pointers), detect a cycle (slow and fast meet), merge two sorted lists.

Binary tree

Each node has at most two children. Depth-first traversals: **preorder** (root, left, right), **inorder** (left, root, right) and **postorder** (left, right, root). Height = 1 + max(height of left, height of right). Level-order traversal uses a queue.

BFS explores level by level: shortest path in an unweighted graph



Level 0: A
Level 1: B, C
Level 2: D, E
Level 3: F

BFS uses a queue. DFS uses a stack or recursion and fits connected components and island counting.

BFS levels on a small graph

Binary search tree (BST)

Left values are smaller and right values are larger. Inorder traversal gives sorted order. Search, insert and delete take O(h) where h is the height.

Graphs

Store a graph as an adjacency list: a dictionary from each node to its neighbours. **BFS** uses a queue and finds the shortest path (fewest edges) in an unweighted graph. **DFS** uses recursion or a stack and finds connected components, cycles and reachability.

Grids as graphs

A grid cell connects to its four neighbours. Counting islands, flood fill and shortest path in a maze are BFS or DFS on the grid. Keep a visited array so each cell is processed once.

Common patterns

Number of connected components: start a DFS from every unvisited node and count the starts. Shortest path in a grid: BFS from the start storing distance. Detect a cycle in an undirected graph: DFS and see a visited neighbour that is not the parent.

Formula and rule box

- Preorder: root, left, right
- Inorder: left, root, right (sorted for a BST)
- Postorder: left, right, root
- Height = 1 + max(h(left), h(right))
- BFS: queue, shortest path in an unweighted graph
- DFS: recursion or stack, components and cycles
- Edges in a tree with n nodes: n − 1

Step-by-step method

1. Choose the structure: list, tree or graph.
2. Pick BFS (shortest path) or DFS (explore everything).
3. Keep a visited set.
4. Handle disconnected graphs by starting from every node.

Solved examples

Example 1. Reverse a linked list and print its values. (Use a simple class.)

Solution: Walk the list, pointing each node back at the previous one.

```python
class Node:
    def __init__(self, v):
        self.v = v
        self.next = None
n = int(input())
vals = list(map(int, input().split()))
head = None
tail = None
for v in vals:
    node = Node(v)
    if head is None:
        head = tail = node
    else:
        tail.next = node
        tail = node
prev = None
cur = head
while cur:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
out = []
while prev:
    out.append(prev.v)
    prev = prev.next
print(*out)
```

Sample input: `5 / 1 2 3 4 5` **Sample output:** `5 4 3 2 1`

Answer: Output: 5 4 3 2 1

Example 2. Given a binary tree in level order (-1 means no node) print its height (number of levels).

Solution: Build children by index: the children of node i are 2i + 1 and 2i + 2. Height = 1 + max(height of children).

```python
n = int(input())
a = list(map(int, input().split()))
def h(i):
    if i >= n or a[i] == -1:
        return 0
    return 1 + max(h(2 * i + 1), h(2 * i + 2))
print(h(0))
```

Sample input: `7 / 1 2 3 4 -1 -1 5` **Sample output:** `3`

Answer: Output: 3

Example 3. Print the shortest number of edges from node 1 to node n in an undirected graph.

Solution: Use BFS: the first time you reach a node is by the shortest path.

```python
from collections import deque
n, m = map(int, input().split())
g = {i: [] for i in range(1, n + 1)}
for _ in range(m):
    u, v = map(int, input().split())
    g[u].append(v)
    g[v].append(u)
dist = {1: 0}
q = deque([1])
while q:
    u = q.popleft()
    for v in g[u]:
        if v not in dist:
            dist[v] = dist[u] + 1
            q.append(v)
print(dist.get(n, -1))
```

```cpp
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) { int u, v; cin >> u >> v; g[u].push_back(v);
g[v].push_back(u); }
    vector<int> d(n + 1, -1); d[1] = 0;
    queue<int> q; q.push(1);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (d[v] < 0) { d[v] = d[u] + 1; q.push(v); }
    }
    cout << d[n] << endl;
    return 0;
}
```

Sample input: `6 7 / 1 2 / 1 3 / 2 4 / 3 5 / 4 6 / 5 6 / 2 3` **Sample output:** `3`

Answer: Output: 3

Example 4. Count the number of islands (connected groups of 1s, four directions) in a grid.

Solution: Run DFS from every unvisited 1 and mark its whole island.

```python
r, c = map(int, input().split())
g = [list(map(int, input().split())) for _ in range(r)]
seen = set()
def dfs(i, j):
    st = [(i, j)]
    while st:
        x, y = st.pop()
        if (x, y) in seen:
            continue
        seen.add((x, y))
        for dx, dy in ((1,0),(-1,0),(0,1),(0,-1)):
            a, b = x + dx, y + dy
            if 0 <= a < r and 0 <= b < c and g[a][b] == 1 and (a, b) not in seen:
                st.append((a, b))
cnt = 0
for i in range(r):
    for j in range(c):
        if g[i][j] == 1 and (i, j) not in seen:
            cnt += 1
            dfs(i, j)
print(cnt)
```

Sample input: `3 4 / 1 1 0 0 / 0 1 0 1 / 1 0 0 1` **Sample output:** `3`

Answer: Output: 3

Shortcuts

- BFS for 'minimum steps', DFS for 'count or explore'.
- A visited set stops infinite loops.
- For trees given as an array, children of i are 2i + 1 and 2i + 2.

Common mistakes

- Forgetting the visited set in BFS/DFS.
- Using DFS when a shortest path is needed.
- Not starting a search from every node in a disconnected graph.

Practice questions: Linked Lists, Trees and Graphs

1. Print the number of nodes of a complete binary tree of height 3 (levels). *Sample input:* 3 — Basic
2. Print the preorder traversal of the tree [1,2,3] with 2 left and 3 right child of 1. *Sample input:* 1 2 3 — Basic
3. Print the middle node value of a linked list with these values (second middle if even). *Sample input:* 1 2 3 4 5 6 — Basic
4. Print YES if the linked list 1→2→3→1 (last points to first) has a cycle. *Sample input:* 1 2 3 — Basic
5. Print the number of connected components of the graph with n = 5 and edges 1-2, 2-3, 4-5. *Sample input:* 5 3 / 1 2 / 2 3 / 4 5 — Medium
6. Print the inorder traversal of the BST built by inserting 5, 3, 8, 1, 4. *Sample input:* 5 3 8 1 4 — Medium
7. Print the shortest number of steps in a 3×3 grid from top-left to bottom-right (no blocks), moving 4 directions. *Sample input:* 3 3 — Medium

8. Print the number of edges in a tree with 10 nodes. *Sample input:* `10` Medium

Answers: see the answer key at the end of this guide.

15 ADVANCED CODING · GOOD TO KNOW

Bit Manipulation

Numbers are stored as bits. Bit tricks give very fast solutions to counting, parity and 'find the unique element' problems, and are the key to subset enumeration for small n.

Start from zero

Binary basics

Decimal 13 is 1101 in binary (8 + 4 + 1). Each bit is a power of two. Python: `bin(13)` returns '0b1101'; C++ can print with `bitset`.

The operators

AND (&): 1 only if both bits are 1. OR (|): 1 if either is 1. XOR (^): 1 if the bits differ. NOT (~) flips bits. Left shift x << k multiplies by 2^k; right shift x >> k divides by 2^k (floor).

Bits of 13 and 6, and the common operators

13	1101
6	0110
13 & 6	0100 = 4
13 | 6	1111 = 15
13 ^ 6	1011 = 11
13 << 1	11010 = 26
13 >> 1	0110 = 6

Bit operations on 13 and 6

Useful tricks

Check the k-th bit: `(x >> k) & 1`. Set a bit: `x | (1 << k)`. Clear: `x & ~(1 << k)`. Toggle: `x ^ (1 << k)`. Odd or even: `x & 1`. Power of two: `x > 0 and (x & (x - 1)) == 0`.

Counting set bits

Brian Kernighan's trick: `x &= x - 1` removes the lowest set bit, so loop until x is 0 and count the steps. Python: `bin(x).count('1')` or `x.bit_count()`.

XOR properties

a ^ a = 0, a ^ 0 = a, and XOR is commutative. So XOR-ing all numbers of an array where every number appears twice except one leaves the unique number. Swap two numbers without a temporary: a ^= b; b ^= a; a ^= b.

Subsets with bitmasks

For n ≤ 20, every number from 0 to $2^n - 1$ represents a subset: bit i tells whether element i is included. Loop over masks and test each bit.

Formula and rule box

- x & 1 → odd or even
- x & (x − 1) removes the lowest set bit
- Power of two: x & (x − 1) == 0 (x > 0)
- a ^ a = 0, a ^ 0 = a
- x << k = x × 2^k; x >> k = x // 2^k
- Subsets of n items: masks 0 ... 2^n − 1

Step-by-step method

1. Write the numbers in binary on paper.
2. Pick the operator that creates the pattern you need.
3. Test small values (0, 1, powers of two).
4. Remember Python's integers are unbounded; mask if needed.

Solved examples

Example 1. Print the number of set bits (1s) in n.

Solution: Use Kernighan's trick: each step clears one set bit.

```python
n = int(input())
cnt = 0
while n:
    n &= n - 1
    cnt += 1
print(cnt)
```

```cpp
#include <iostream>
using namespace std;
int main() {
    long long n; cin >> n;
    int cnt = 0;
    while (n) { n &= n - 1; cnt++; }
    cout << cnt << endl;
    return 0;
}
```

Sample input: 255 **Sample output:** 8

Answer: Output: 8

Example 2. Print YES if n is a power of two.

Solution: A power of two has exactly one set bit, so n & (n − 1) is 0.

```python
n = int(input())
print('YES' if n > 0 and n & (n - 1) == 0 else 'NO')
```

Sample input: 64 **Sample output:** YES

Answer: Output: YES

Example 3. Every number in the array appears twice except one. Print the single number.

Solution: XOR cancels equal pairs.

```python
n = int(input())
a = list(map(int, input().split()))
r = 0
for x in a:
    r ^= x
print(r)
```

Sample input: `7 / 4 1 2 1 2 4 9` **Sample output:** `9`

Answer: Output: 9

Example 4. Print all subsets of an array of n numbers whose sum equals S using bitmasks. Print the count.

Solution: Each mask from 0 to $2^n - 1$ picks the elements whose bit is 1.

```python
n, S = map(int, input().split())
a = list(map(int, input().split()))
cnt = 0
for mask in range(1 << n):
    s = 0
    for i in range(n):
        if mask >> i & 1:
            s += a[i]
    if s == S:
        cnt += 1
print(cnt)
```

Sample input: `4 10 / 2 3 5 7` **Sample output:** `2`

Answer: Output: 2

Shortcuts

- x & (x − 1) is the most useful identity.
- Use 1 << i to test or set bit i.
- XOR finds the unpaired element in one pass.

Common mistakes

- Using ^ as 'power' (it is XOR).
- Forgetting brackets: (x >> i) & 1.
- Using 1 << i with int overflow in C++ for i ≥ 31.

Practice questions: Bit Manipulation

1. Print the binary form of 10. *Sample input:* `10` Basic
2. Print the number of set bits of n. *Sample input:* 29 Basic
3. Print YES if n is odd using bit operations. *Sample input:* 15 Basic
4. Print the value of n after setting the 3rd bit (0-indexed). *Sample input:* 5 Basic
5. Print the XOR of all numbers from 1 to n. *Sample input:* 7 Medium
6. Print the number of different bit positions between 10 and 4. *Sample input:* `10 4` Medium
7. Print the next power of two that is ≥ n. *Sample input:* 37 Medium

8. Print two numbers appearing once in [1,2,3,2,1,4] as 'a b' (smaller first). *Sample input:* 6 / 1 2 3 2 1 4 **Medium**

Answers: see the answer key at the end of this guide.

16 ADVANCED CODING · MUST-DO

Exam Strategy, Edge Cases and Debugging

Technique wins marks in the coding round as much as knowledge. A calm plan for the 90 minutes, a habit of testing edge cases and a clean way to debug will save you from the most common failures.

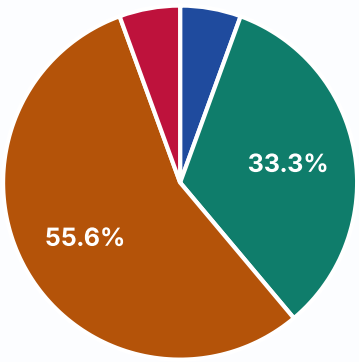
Start from zero

Plan your 90 minutes

Spend 5 minutes reading both problems. Solve the Easy problem first (about 30–35 minutes): it usually has a direct approach. Then take the Medium problem (about 50 minutes). Keep the last 5 minutes for testing. Always submit something that passes the sample tests before polishing.

Brute force first

Write the simplest correct solution first, even if it is slow. It scores partial marks on small hidden tests. Then look for the repeated work and optimise (prefix sums, hash map, sorting, DP). A slow correct answer beats a fast wrong one.



- Read and plan (5 min)
- Easy problem (30 min)
- Medium problem (50 min)
- Test and debug (5 min)

90 minutes in total

Easy ≈ 35 min, Medium ≈ 55 min

Always submit something that passes the sample tests.

A sensible split of the 90 minutes

Edge-case checklist

Empty or single-element input; all elements equal; already sorted or reverse sorted; zeros and negative numbers; the maximum size and maximum values (overflow); duplicates; strings with spaces or one character; k larger than n.

Debugging method

Run the sample by hand and compare line by line. Print intermediate values with small input. Check index ranges and off-by-one errors. Compare your fast solution with a brute force on random small tests. Remove debug prints before submitting.

Common runtime failures

Reading input in the wrong format; recursion depth exceeded; integer overflow in C++; list index out of range; dividing by zero; very slow input handling (use fast input for large data); printing extra spaces or text.

Templates to keep ready

Fast input reader, a gcd function, a sieve, a binary search, a BFS skeleton and a DP table skeleton. Practise writing each from memory in your chosen language until they take under two minutes.

Formula and rule box

- Time plan: 5 min read + 30–35 min Easy + 50 min Medium + 5 min test
- Brute force first, then optimise
- Test: empty, one element, all equal, maximum, negatives
- Overflow: use long long in C++
- Always match the output format exactly

Step-by-step method

1. Read both problems and choose the order.
2. Write brute force, pass the sample.
3. Test the edge cases from the checklist.
4. Optimise only if the constraints demand it.

Solved examples

Example 1. Print the maximum of an array safely when the array may be empty (print 0 if empty).

Solution: Check for the empty case before calling max().

```python
n = int(input())
a = list(map(int, input().split())) if n > 0 else []
print(max(a) if a else 0)
```

Sample input: `0` **Sample output:** `0`

Answer: Output: 0

Example 2. Print the sum of an array with large values (up to 10^9 each, n up to 10^5). Python is safe; in C++ use long long.

Solution: The sum can reach 10^{14}, which does not fit in a 32-bit int.

```python
n = int(input())
a = list(map(int, input().split()))
print(sum(a))
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int n; cin >> n;
    long long s = 0, x;
    for (int i = 0; i < n; i++) { cin >> x; s += x; }
    cout << s << endl;
    return 0;
}
```

Sample input: `3 / 1000000000 1000000000 1000000000` **Sample output:** `3000000000`

Answer: Output: 3000000000

Example 3. Print the average of the numbers rounded to 2 decimal places.

Solution: Mind the output format: use formatting, and avoid dividing by zero.

```python
n = int(input())
a = list(map(int, input().split()))
print('%.2f' % (sum(a) / n))
```

Sample input: `4 / 1 2 3 5` **Sample output:** `2.75`

Answer: Output: 2.75

Example 4. Read until the end of input: print the number of lines.

Solution: When the number of lines is not given, read all of standard input.

```python
import sys
data = sys.stdin.read().split('\n')
print(len([x for x in data if x.strip()]))
```

Sample input: `a / b / c` **Sample output:** `3`

Answer: Output: 3

Shortcuts

- Always submit a working partial solution early.
- Test with the smallest and largest possible input.
- Keep templates in your head: fast input, gcd, sieve, binary search, BFS, DP.

Common mistakes

- Starting with the hard problem and finishing nothing.
- Not testing on edge cases.
- Spending 60 minutes on the Easy problem.

Practice questions: Exam Strategy, Edge Cases and Debugging

1. How should you split the 90 minutes between the two problems? Basic
2. Which problem should you start with? Basic
3. You are stuck on the Medium problem. What do you submit? Basic
4. Name five edge cases to test. Basic
5. A C++ answer is wrong only on large tests. What is the first suspect? Medium
6. Why is your Python code too slow at $n = 10^6$? Medium
7. What should you print for YES/NO problems? Medium
8. What do you do in the last 5 minutes? Medium

Answers: see the answer key at the end of this guide.

Cheat sheet

Every key rule and formula of the section on a few pages. Revise it daily in the last two weeks.

1. Programming Fundamentals and Input/Output

- int ≈ ±2.1 × 10^9; long long ≈ ±9.2 × 10^{18}
- Use long long when a product can exceed 2 × 10^9
- (a × b) mod m = ((a mod m) × (b mod m)) mod m
- Multiple tests: read T, then loop T times
- Digit sum: repeat n % 10 and n // 10
- Leap year: divisible by 4 and (not by 100 or divisible by 400)

2. Time Complexity and Constraints

- ≈10^8 operations/second (C++), ≈10^7 (Python)
- n ≤ 10^5 → O(n log n) or better
- n ≤ 5,000 → O(n^2) is fine
- n ≤ 20 → O(2^n) is fine
- sort = O(n log n); binary search = O(log n)
- dict/set lookup = O(1) average

3. Arrays

- P[i] = a[0] + ... + a[i−1]; sum(l..r) = P[r+1] − P[l]
- Kadane: cur = max(x, cur + x); best = max(best, cur)
- Missing number from 1..n: n(n+1)/2 − sum
- Rotate left by k: a[k:] + a[:k] (k %= n)
- Sum of first n numbers: n(n+1)/2

4. Strings

- s[::-1] reverses a string
- Anagram ⇔ equal character counts
- Number of substrings = n(n + 1)/2
- ord('a') = 97; ord(c) − 97 gives 0–25
- Palindrome: s == s[::-1]
- Join pieces: ''.join(list)

5. Math and Number Theory

- Prime test: check divisors up to √n
- gcd(a, b) = gcd(b, a mod b)
- lcm(a, b) = a × b / gcd(a, b)
- pow(a, b, m) computes a^b mod m
- Divisors of p^a q^b: (a + 1)(b + 1)
- Trailing zeros of n! = n//5 + n//25 + n//125 + ...
- Sum 1..n = n(n + 1)/2

6. Sorting and Searching

- Sorting: O(n log n)
- Binary search: O(log n)
- mid = lo + (hi − lo) // 2
- count(x) = bisect_right − bisect_left
- Sort key: (−marks, name) for marks descending
- Searching on the answer: check(mid) is monotone

7. Hashing, Sets and Maps

- Membership in set/dict: O(1) average
- Frequency: d[x] = d.get(x, 0) + 1
- Two-sum: look for target − x in seen
- Subarray sum K: count[prefix − K]
- Group key for anagrams: sorted(word)
- Counter(a).most_common(1) gives the most frequent item

8. Two Pointers and Sliding Window

- Pair sum on sorted array: i from the left, j from the right
- Window sum: new = old + a[right] − a[left]
- Longest unique substring: move left past the last duplicate
- Time: O(n) because each pointer moves at most n times
- Smallest subarray with sum ≥ S: shrink while sum ≥ S

9. Recursion and Backtracking

- Base case first, then a smaller recursive call
- Fibonacci with memo: O(n)
- Subsets: 2^n; permutations: n!
- Tower of Hanoi moves: 2^n − 1
- Backtracking: choose → explore → unchoose
- N-Queens (n = 4): 2 solutions; (n = 8): 92

10. Stack and Queue

- Stack: LIFO; queue: FIFO
- Python stack: list.append / list.pop
- Python queue: collections.deque, popleft()
- Balanced brackets ⇔ stack empty at the end
- Next greater element: O(n) with a monotonic stack
- Postfix: operand order matters for − and ÷

11. Matrix and 2D Arrays

- Main diagonal: i == j; anti-diagonal: i + j == n − 1
- Rotate 90° clockwise = transpose + reverse each row
- Four neighbours: (±1, 0), (0, ±1)
- Spiral: top → right → bottom → left, shrink boundaries
- Sorted matrix search: start at the top-right corner

12. Greedy Algorithms

- Activity selection: sort by end time
- Fractional knapsack: sort by value/weight
- Greedy coin change works only for canonical systems
- Jump game: reach = max(reach, i + a[i])
- Minimum platforms: sort arrivals and departures

13. Dynamic Programming

- Climbing stairs: dp[n] = dp[n−1] + dp[n−2]
- Coin change: dp[a] = 1 + min(dp[a − c])
- LCS: match → dp[i−1][j−1] + 1, else max of neighbours
- 0/1 knapsack: dp[w] = max(dp[w], dp[w − wt] + val)
- LIS: dp[i] = 1 + max dp[j] with a[j] < a[i]
- Grid paths: dp[i][j] = dp[i−1][j] + dp[i][j−1]

14. Linked Lists, Trees and Graphs

- Preorder: root, left, right
- Inorder: left, root, right (sorted for a BST)
- Postorder: left, right, root
- Height = 1 + max(h(left), h(right))
- BFS: queue, shortest path in an unweighted graph
- DFS: recursion or stack, components and cycles
- Edges in a tree with n nodes: n − 1

15. Bit Manipulation

- x & 1 → odd or even
- x & (x − 1) removes the lowest set bit
- Power of two: x & (x − 1) == 0 (x > 0)
- a ^ a = 0, a ^ 0 = a
- x << k = x × 2^k; x >> k = x // 2^k
- Subsets of n items: masks 0 … 2^n − 1

16. Exam Strategy, Edge Cases and Debugging

- Time plan: 5 min read + 30–35 min Easy + 50 min Medium + 5 min test
- Brute force first, then optimise
- Test: empty, one element, all equal, maximum, negatives
- Overflow: use long long in C++
- Always match the output format exactly

Code templates to practise

Write each of these from memory until it takes under two minutes.

Python: fast input

```
import sys
input = sys.stdin.readline          # fast input
n = int(input())
a = list(map(int, input().split()))
```

Python: imports to remember

```
import math
print(math.gcd(12, 18))             # 6
print(pow(2, 10, 10**9 + 7))        # fast power with modulo
from collections import Counter, deque
from functools import lru_cache
import bisect
```

C++: standard start

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());
    return 0;
}
```

Python: binary search

```
# binary search template
lo, hi = 0, n - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == x: return mid
    if a[mid] < x: lo = mid + 1
    else: hi = mid - 1
```

Python: BFS

```
# BFS template
from collections import deque
q = deque([start]); seen = {start}
while q:
    u = q.popleft()
    for v in g[u]:
        if v not in seen:
            seen.add(v); q.append(v)
```

Python: DP table

```
# DP skeleton
dp = [base] * (n + 1)
for i in range(1, n + 1):
    dp[i] = best_of(dp[i - 1], dp[i - 2], ...)
```

Edge-case checklist

Test this	Why it breaks solutions
n = 0 or n = 1	Loops and max() fail on empty or one-element arrays
All elements equal	Strict comparisons skip duplicates
Sorted and reverse sorted	Worst case for some algorithms
Negative numbers and zeros	Kadane, products and division behave differently
Maximum values	Overflow in int; time limit
k > n or k = 0	Rotation and window problems
Strings with spaces, one character, mixed case	Palindrome, anagram and word counts
Duplicates	Sets and maps change counts
Disconnected graph or empty grid	BFS/DFS must start from every node

Submission checklist

- Sample input gives the sample output exactly
- No debug prints left
- Handled T test cases if the format has them
- Used long long or Python where needed
- Edge cases tested
- Time limit estimated from the constraints

Mini test: 20 questions in 25 minutes

Quick-fire revision: answer each item in your head or on paper. Target: 16 or more correct.

1. Print the sum of digits of n. Input: 9045. (Write the answer.)
2. Is the year 2024 a leap year? (YES/NO)
3. What complexity is needed for n = 10^5? (write the target)
4. Find the second largest distinct number in 5 9 1 9 7.
5. Rotate [1,2,3,4,5] left by 2. Write the result.
6. Maximum subarray sum of [-2, 1, -3, 4, -1, 2, 1, -5, 4].
7. Is 'A man, a plan, a canal: Panama' a palindrome (ignoring case and punctuation)?
8. Number of primes up to 30.
9. gcd(48, 18) and lcm(48, 18).
10. Index of 23 in the sorted array [2,5,8,12,16,23,38].
11. Indices of two numbers summing to 9 in [2,7,11,15].
12. Maximum sum of 2 consecutive elements in [1,4,2,10,2].
13. Number of subsets of a 4-element set.
14. Is '{[()]}' balanced?
15. Spiral order of the matrix [[1,2],[3,4]].
16. Minimum coins for 6 with coins 1, 3, 4 (DP).
17. Length of the LIS of [10, 9, 2, 5, 3, 7, 101, 18].
18. Shortest steps from node 1 to node 4 in the graph 1-2, 2-3, 3-4, 1-3.
19. Number of set bits in 255.
20. Single number in [4, 1, 2, 1, 2].

Answer key

1. Programming Fundamentals and Input/Output

1. Output: 19. Hint: Repeatedly take n % 10 and n // 10.

2. Output: NO. Hint: Divisible by 4 and (not by 100 or divisible by 400).

3. Output: 5050. Hint: Use n(n+1)/2 instead of a loop.

4. Output: 21. Hint: Reverse the string and convert back to int.

5. Output: 5. Hint: Convert to string or divide repeatedly by 10.

6. Output: 19. Hint: Use max() or compare pairwise.

7. Output: 146326063. Hint: Multiply and take the modulo at each step (needed in C++).

8. Output: 9. Hint: n//3 + n//5 − n//15.

2. Time Complexity and Constraints

1. O(n log n) or better

2. About 10^{10}: too slow

3. Hash set or dictionary

4. O(n log n)

5. O(log n)

6. $O(2^n)$ is fine (about 10^6 subsets)

7. Each call shifts all elements: O(n) each, so $O(n^2)$ in total

8. Prefix sums

3. Arrays

1. Output: 7. Hint: Use a set, sort, and take the second last.

2. Output: 3. Hint: Brute force is fine for small n; use a map for large n.

3. Output: 1 3 12 4 0 0 0. Hint: Collect non-zeros, then append zeros.

4. Output: 30. Hint: Sort: compare two smallest with two largest.

5. Output: 17 5 2. Hint: Scan from the right, keep the maximum so far.

6. Output: 4 5 1 2 3. Hint: Right rotation by k equals left rotation by n − k.

7. Output: 4. Hint: Keep a counter that resets when the order breaks.

8. Output: 3. Hint: Use a set to detect the first repeat.

4. Strings

1. Output: 3. Hint: Count characters in 'aeiouAEIOU'.

2. Output: olleh dlrow. Hint: Split, reverse each, join.

3. Output: a. Hint: Count with a dictionary, sort by (−count, char).

4. Output: 4. Hint: Compare characters from the left until they differ.

5. Output: YES. Hint: Use a set of letters.

6. Output: 6. Hint: split() handles multiple spaces.

7. Output: abc def. Hint: Use (ord(c) − 97 + 3) % 26.

8. Output: 3. Hint: Use a sliding window with a set or last-seen map.

5. Math and Number Theory

1. Output: 15. Hint: Use a sieve.

2. Output: 55. Hint: Iterate with two variables.

3. Output: 12. Hint: Fold math.gcd over the list.

4. Output: YES. Hint: Sum each digit raised to the number of digits.

5. Output: 9. Hint: Loop to √n and count in pairs.

6. Output: 101101. Hint: Divide by 2 repeatedly and read remainders upwards.

7. Output: YES. Hint: Sum of proper divisors equals n.

8. Output: 24. Hint: Add n//5 + n//25 + n//125 + ...

6. Sorting and Searching

1. Output: 12. Hint: Sort distinct values in descending order.

2. Output: 1 3 4 7 9. Hint: Use the built-in sort.

3. Output: 2. Hint: After sorting, only neighbours matter.

4. Output: 3. Hint: bisect_left.

5. Output: 5. Hint: Sort and take the middle.

6. Output: YES. Hint: Compare the length of the list and the set.

7. Output: 0. Hint: Binary search: compare mid with the last element.

8. Output: 9. Hint: Search the largest x with x·x ≤ n.

7. Hashing, Sets and Maps

1. Output: 5. Hint: Keep a set of seen values.

2. Output: YES. Hint: Compare Counter(a) and Counter(b).

3. Output: 4. Hint: len(set(a)).

4. Output: 4. Hint: Counter and a tie-break.

5. Output: 4. Hint: Add c(c−1)/2 for each frequency c.

6. Output: 6. Hint: Prefix-sum map: count equal prefix sums.

7. Output: 4. Hint: Set plus counting from run starts.

8. Output: 3. Hint: Group by sorted word.

8. Two Pointers and Sliding Window

1. Output: 4. Hint: Slow pointer writes new values.

2. Output: 12. Hint: Window of size 2.

3. Output: 3. Hint: Two pointers with care for equal values.

4. Output: 3. Hint: Window plus a count map.

5. Output: YES. Hint: i from the left, j from the right.

6. Output: 3. Hint: Reset the counter on 0.

7. Output: 2. Hint: Shrink from the left while valid.

8. Output: 3. Hint: Variable window.

9. Recursion and Backtracking

1. Output: 720. Hint: n × factorial(n − 1), base case 1.

2. Output: 6765. Hint: Memoise or iterate.

3. Output: 32. Hint: Each element has two choices.

4. Output: 63. Hint: T(n) = 2T(n−1) + 1.

5. Output: 24. Hint: n!.

6. Output: 10. Hint: Backtracking with column and diagonal sets.

7. Output: 2. Hint: Include/skip recursion with a running sum.

8. Output: 10. Hint: Recursion with memo, or C(r + c − 2, r − 1).

10. Stack and Queue

1. Output: 6. Hint: Stack of indices.

2. Output: NO. Hint: Check the top on each closer.

3. Output: 1 1 0 2 1 0. Hint: Monotonic stack of indices.

4. Output: 48. Hint: Push numbers, apply operators.

5. Output: 1 4. Hint: Last in, first out.

6. Output: 3 3 5 5. Hint: Deque of candidate indices.

7. Output: kcats. Hint: Push all, pop all.

8. Output: 2. Hint: Count unmatched openers and closers.

11. Matrix and 2D Arrays

1. Output: 21. Hint: Two nested loops.

2. Output: 1 4 / 2 5 / 3 6. Hint: zip(*g).

3. Output: 40. Hint: First/last row or column.

4. Output: 24. Hint: Sum each row, take the max.

5. Output: YES. Hint: Compare g[i][j] and g[j][i].

6. Output: 11. Hint: j > i.

7. Output: 3. Hint: DFS/BFS flood fill.

8. Output: 7. Hint: DP: dp[i][j] = grid + min(top, left).

12. Greedy Algorithms

1. Output: 4. Hint: Largest coin first.

2. Output: 3. Hint: Sort by end time.

3. Output: 1. Hint: Sort both lists and match smallest to smallest.

4. Output: 3. Hint: Track the current range end and farthest reach.

5. Output: 240. Hint: Sort by value per weight.

6. Output: 6. Hint: Sort both and pair in order.

7. Output: 5. Hint: Track the minimum price so far.

8. Output: 132219. Hint: Remove the first digit that is larger than the next one.

13. Dynamic Programming

1. Output: 89. Hint: Same as Fibonacci.

2. Output: 12. Hint: dp[i] = max(dp[i−1], dp[i−2] + a[i]).

3. Output: 10. Hint: dp[i][j] = dp[i−1][j] + dp[i][j−1].

4. Output: YES. Hint: Subset-sum DP over sums.

5. Output: 3. Hint: dp[a] = 1 + min(dp[a − c]).

6. Output: 3. Hint: dp[i] = 1 + max dp[j].

7. Output: 3. Hint: dp[i][j] = min(insert, delete, replace).

8. Output: 4. Hint: Loop coins outside, amounts inside.

14. Linked Lists, Trees and Graphs

1. Output: 7. Hint: 2^h − 1.
2. Output: 1 2 3. Hint: Root, left, right.
3. Output: 4. Hint: Slow and fast pointers.
4. Output: YES. Hint: Slow and fast pointers meet.
5. Output: 2. Hint: DFS from each unvisited node.
6. Output: 1 3 4 5 8. Hint: Inorder of a BST is sorted.
7. Output: 4. Hint: BFS distance.
8. Output: 9. Hint: n − 1.

15. Bit Manipulation

1. Output: 1010. Hint: Divide by 2 repeatedly.
2. Output: 4. Hint: Kernighan's trick.
3. Output: YES. Hint: n & 1.
4. Output: 13. Hint: n | (1 << 3).
5. Output: 0. Hint: Pattern repeats every 4.
6. Output: 3. Hint: Count set bits of the XOR.
7. Output: 64. Hint: Use bit_length.
8. Output: 3 4. Hint: XOR all, split by a set bit.

16. Exam Strategy, Edge Cases and Debugging

1. About 5 min reading, 30–35 min Easy, about 50 min Medium, and 5 min testing
2. The Easy one; submit a working solution before the Medium
3. The brute-force solution that passes the sample and small tests
4. Empty, one element, all equal, maximum size, negative numbers
5. Integer overflow: use long long
6. Nested loops or slow input: use O(n) logic and sys.stdin
7. Exactly the capitalisation shown in the sample
8. Test edge cases, remove debug prints and submit

Mini test answers

1. 18
2. YES
3. O(n log n) or better
4. 7
5. 3 4 5 1 2
6. 6
7. YES
8. 10
9. 6 and 144
10. 5
11. 0 1
12. 12
13. 16
14. YES
15. 1 2 4 3
16. 2
17. 4
18. 2
19. 8
20. 4